

Zaawansowane programowanie

Kierunek: Bioinformatyka stopień II, semestr I	Data oddania: 15.06.2025 r.
Przedmiot: Zaawansowane programowanie	Prowadzący zajęcia: prof. dr hab. inż. Marta Kasprzak, dr inż. Marcin Radom
Przygotowała: Emilia Bacik 155643	Grupa laboratoryjna: 2

Spis treści:

1. Opis problemu wybranego w ramach projektu i wybranej metaheurystyki, która ma go rozwiązać,
2. Szczegółowy opis zaimplementowanego rozwiązania:
 - a) Generator instancji wraz z mechanizmem wprowadzania błędów,
 - b) Generowanie populacji początkowej,
 - c) Selekcja osobników do krzyżowania i pozostania w populacji w kolejnym pokoleniu,
 - d) Krzyżowanie i mechanizm naprawczy,
 - e) Mutacje,
 - f) Sprawdzenie najlepszego osobnika i aktualizacja interfejsu,
3. Opis interfejsu aplikacji okienkowej,
4. Opis procesu strojenia parametrów,
5. Testy podsumowujące jakość rozwiązania,
6. Złożoność obliczeniowa i pamięciowa.

1. Opis problemu wybranego w ramach projektu i wybranej metaheurystyki, która ma go rozwiązać

Problemem wybranym w ramach niniejszego projektu do rozwiązania jest problem VII - MIN LENGTH ALIGNMENT. Instancją problemu jest znakowa macierz $M_{m \times n}$ zawierająca m sekwencji nukleotydowych o długości n , natomiast rozwiązaniem dopasowanie globalne m sekwencji takie, że w żadnej kolumnie dopasowania nie ma konfliktu znaków (nie licząc spacji) i długość dopasowania jest minimalna. Jest to więc problem minimalizacji. Funkcja celu w przypadku tego problemu to po prostu długość dopasowania. Ważnym aspektem jest ograniczona liczba rozwiązań dopuszczalnych, duża część przestrzeni możliwych rozwiązań będzie niedopuszczalna z uwagi na warunek o braku konfliktu znaków w każdej kolumnie dopasowania. Motywacją do wyboru tego problemu było jego praktyczne zastosowanie w bioinformatyce (porównywanie sekwencji nukleotydowych pozwala między innymi określić pokrewieństwo i ewolucyjne pochodzenie między osobnikami/gatunkami, a także znaleźć strukturę i funkcję nowo zsekwencjonowanych cząsteczek RNA), prosta do określenia funkcja celu (jasna, łatwo porównywalna ocena jakości wyników) oraz duża ilość rozwiązań niedopuszczalnych, co wydawało się ciekawym wyzwaniem.

Wybraną w ramach niniejszego projektu w celu rozwiązania tego problemu metaheurystyką jest algorytm genetyczny. Jego zaletą jest zdolność do efektywnej eksploracji dużych i złożonych przestrzeni rozwiązań oraz możliwość jednoczesnego przeszukiwania wielu

obszarów przestrzeni dopuszczalnych (dzięki działaniu na populacji rozwiązań, nie pojedynczym jak tabu search). Dodatkowo mechanizmy krzyżowania i mutacji pozwalają na łączenie korzystnych cech różnych rozwiązań, co może ułatwić odnajdywanie krótkich dopasowań spełniających warunek braku konfliktów znaków. Te cechy wydawały się szczególnie korzystne w kontekście problemu dopasowania sekwencji, ponieważ charakteryzuje się on bardzo specyficznym chromosomem (bardzo ważna jest między innymi kolejność nukleotydów w dopasowaniu) i różne fragmenty optymalnego rozwiązania mogą leżeć daleko od siebie w przestrzeni rozwiązań. W przypadku algorytmu operującego na pojedynczym rozwiązaniu szczególnie trudne mogłoby być wydostanie się z optimum lokalnego (proces "wymiany miejscami" oddalonych od siebie nukleotydów może być trudny do przeprowadzenia gdy po drodze znajduje się wiele rozwiązań niedopuszczalnych), natomiast algorytm operujący na populacji, z większym udziałem elementu losowości może dla różnych osobników odnaleźć różne fragmenty rozwiązania optymalnego i na drodze krzyżowania łatwiej połączyć je w jedno, krótkie rozwiązanie.

W tym projekcie jako sekwencje nukleotydowe rozważane są tak naprawdę sekwencje DNA, to znaczy złożone z nukleotydów "A", "C", "G", "T", aczkolwiek dostosowanie go w razie potrzeby do sekwencji RNA byłoby bardzo szybkie, ponieważ rodzaje nukleotydów nie wpływają w żaden sposób na działanie metaheurystyki.

2. Szczegółowy opis zaimplementowanego rozwiązania

a) Generator instancji wraz z mechanizmem wprowadzania błędów

Zaimplementowane rozwiązanie zawiera dwa sposoby przygotowania instancji: wpisanie jej do pola tekstowego (TextBox) lub wygenerowanie podając (lub nie) parametry takie jak długość rozwiązania optymalnego (na podstawie którego zostanie wygenerowana instancja), ilość sekwencji w instancji i ich długość, a także liczba błędów (w odniesieniu do wygenerowanego rozwiązania optymalnego). Taki sposób generowania instancji z rozwiązania optymalnego pozwala na znaną (w przypadku wersji instancji bez błędów często pewną) długość rozwiązania optymalnego, dzięki czemu łatwo można ocenić jakość rozwiązania zwróconego przez algorytm genetyczny.

W przypadku wprowadzenia własnej instancji, aplikacja sprawdza jej poprawność, to znaczy czy wszystkie sekwencje są tej samej długości, czy znajdują się w kolejnych liniach pola tekstowego, i czy złożone są ze znaków "A", "G", "C", "T".

W przypadku generowania instancji, aplikacja sprawdza czy wprowadzone wartości parametrów generatora są poprawne (są liczbami). Jeżeli dla któregoś (lub wszystkich) parametru nie zostanie wprowadzona żadna wartość, zostanie ona wylosowana odpowiednio: z zakresu od 2 do 50 dla ilości sekwencji w instancji (m), z zakresu od 5 do 80 dla długości sekwencji w instancji (n), z zakresu od n do $n+50$ dla długości sekwencji optymalnej (sekwencje w instancji nie mogą być dłuższe niż ich dopasowanie), z zakresu od 0 do $3*m$ dla ilości błędów. Sekwencja optymalna jest generowana jako proste losowanie jednego z czterech nukleotydów (z równym prawdopodobieństwem) ponawiane tak długo, aż sekwencja optymalna nie osiągnie odpowiedniej długości. Następnie m razy tworzona jest kopia sekwencji optymalnej, z której usuwane są losowo wybrane z ciągu znaki tak długo, aż osiągnie długość n . Tak otrzymana sekwencja dodawana jest do instancji. Jeżeli ilość błędów jest większa niż 0, do wygenerowanej instancji wprowadzane są losowe modyfikacje sekwencji. Dla każdego błędu losowana jest sekwencja, która ma zostać zmieniona, a następnie rodzaj błędu. Dostępne są dwa typy błędów: substytucja

pojedynczego nukleotydu oraz zamiana miejscami dwóch sąsiadujących nukleotydów. W przypadku substytucji losowany jest nukleotyd (a właściwie jego pozycja) w sekwencji oraz nowa wartość, która zastępuje poprzedni znak. W przypadku zamiany miejscami losowana jest pozycja w sekwencji, a następnie nukleotyd znajdujący się na tej pozycji jest zamieniany z kolejnym nukleotydem. Generator operuje na liczbie pozostałych do wprowadzenia mutacji, która pierwotnie wynosi liczbę błędów (parametr) i jest modyfikowana w zależności od sukcesu lub niepowodzenia próby wprowadzenia mutacji. W celu uniknięcia wielokrotnego modyfikowania tych samych pozycji prowadzona jest lista wszystkich wcześniej zmienionych miejsc. Jeżeli wylosowana operacja nie powoduje rzeczywistej zmiany sekwencji lub dotyczy pozycji, która została już wcześniej zmodyfikowana, jest ona odrzucana, a liczba planowanych błędów zostaje odpowiednio zwiększona tak, aby ostatecznie uzyskać zadaną liczbę skutecznie wprowadzonych błędów. Dodatkowo operacja zamiany sąsiednich nukleotydów traktowana jest jako modyfikacja dwóch pozycji jednocześnie, dlatego po jej wykonaniu licznik pozostałych do wygenerowania błędów jest zmniejszany o 2. Dzięki temu generowane błędy są rozłożone na różnych pozycjach sekwencji i prowadzą do rzeczywistych zmian w instancji, a ich ilość dokładnie odpowiada zadanej w formie parametru ilości.

Ważne uwagi odnośnie długości rozwiązania optymalnego z generatora: w przypadku instancji bez błędów rozwiązanie optymalne z generatora może ale nie musi być rozwiązaniem optymalnym. Prawdziwe rozwiązanie optymalne na pewno jest maksymalnie tak samo długie jak rozwiązanie optymalne z generatora, natomiast może być krótsze. Na przykład w sytuacji, gdy w każdej sekwencji z instancji w momencie tworzenia jej z rozwiązania optymalnego zostanie usunięty ten sam nukleotyd (z tej samej pozycji), przez co dopasowanie będzie dopuszczalnym również bez niego. W sytuacji instancji z błędami prawdziwe rozwiązanie optymalne praktycznie nigdy nie będzie tym z generatora, może być krótsze lub dłuższe. W ramach luźnego oszacowania można przyjąć, że ta różnica powinna być maksymalnie tak duża jak liczba błędów - natomiast jest to bardzo niepewne założenie - mała ilość błędów może dużo mocniej zmienić rozwiązanie, jeżeli np. były to nukleotydy, które rozdzielały podobne do siebie motywy i teraz te motywy mogą zostać do siebie dopasowane lokalnie i stworzyć krótsze dopasowanie globalne. Trudniej wyobrazić sobie sytuację odwrotną, w której mała ilość błędów powoduje drastyczne wydłużenie rozwiązania optymalnego, natomiast w ramach niniejszego projektu nie zostało dowiedzione, że taka sytuacja nie może wystąpić.

b) Generowanie populacji początkowej

Generowanie populacji początkowej odbywa się w oparciu o sekwencje z instancji, z dużym udziałem losowości. Ilość generowanych osobników jest określona przez parametr, użytkownik aplikacji może ją ustawić lub skorzystać z domyślnej wartości. Dla każdego generowanego osobnika ustawiane są liczniki (lista 0 w ilości równej ilości sekwencji w instancji), które będą opisywać pokrycie instancji przez osobnika. W pierwszym kroku losowana jest dowolna sekwencja z instancji i do osobnika dodawany jest jej pierwszy element. Licznik dla tej sekwencji i każdej innej, która ma na pierwszej pozycji ten sam nukleotyd zwiększa się o 1 (i wynosi 1). W następnym kroku znowu losowana jest sekwencja i do osobnika dodawany jest jej element z pozycji, którą określa licznik (czyli gdyby wylosowała się znowu ta sama, zostanie wzięty kolejny nukleotyd, nie ten sam). Dla wszystkich sekwencji dla których na pozycji, określanej przez licznik, znajduje się ten sam nukleotyd co wybrany, zwiększa się licznik. Ten proces jest powtarzany tak długo aż

powstający osobnik pokryje całą instancję (to znaczy wszystkie liczniki wyniosą liczbę równą długości sekwencji) i jego chromosom stanie się rozwiązaniem dopuszczalnym. Wtedy jest on dodawany do populacji. Dla wszystkich osobników populacji początkowej sprawdzana jest od razu funkcja celu i zapisywany jest aktualnie najlepszy pod tym kątem. Funkcja celu osobnika to po prostu długość jego rozwiązania (chromosomu), jeżeli kilka osobników ma tę samą wartość funkcji celu, wszystkie zostaną zapisane i wyświetlone, natomiast tylko dla pierwszego z nich zostanie rozrysowane całe dopasowanie (to znaczy rozpisane wszystkie elementy instancji - więcej informacji o sposobie wyświetlania rozwiązania optymalnego znajduje się w części 3. [Opis interfejsu aplikacji okienkowej](#)). Po wygenerowaniu pokolenia początkowego rozpoczyna się właściwa część algorytmu genetycznego.

c) Selekcja osobników do krzyżowania i pozostania w populacji w kolejnym pokoleniu

W każdej z iteracji (których ilość określa specjalny parametr) najpierw losowane są osobniki, które przeżywają, tzn. zostają dodane do kolejnego pokolenia. Ich ilość jest określana na podstawie parametru, który przechowuje informację o procencie przeżywalności. Selekcja osobników do pozostania w populacji i selekcja rodziców wykorzystywanych podczas krzyżowania zachodzi w ten sam sposób i jest oparta o metodę rankingową z przeskalowanymi wartościami. Osobniki są najpierw sortowane rosnąco względem wartości funkcji celu, czyli długości reprezentowanego dopasowania. Każdemu osobnikowi przypisywana jest następnie waga określająca prawdopodobieństwo jego wyboru jako rodzica. Wagi wyznaczone są liniowo na podstawie parametru presji reprodukcyjnej (wartość większa lub równa 1). Najlepszy osobnik otrzymuje wagę równą wartości tego parametru, najgorszy wagę równą 1, a pozostałe osobniki wartości pośrednie. Dzięki temu lepsze rozwiązania mają większą szansę przekazania swoich cech potomstwu, jednak również słabsze osobniki mogą zostać wybrane, co pozwala zachować różnorodność populacji i ogranicza ryzyko utknięcia algorytmu w optimum lokalnym. Po wyznaczeniu wag obliczana jest ich suma, a następnie losowana jest liczba rzeczywista z przedziału od 0 do tej sumy. Na podstawie tej wartości wykonywany jest wybór.

d) Krzyżowanie i mechanizm naprawczy

Mechanizm krzyżowania zachodzi tak długo, aż rozmiar nowej populacji nie będzie odpowiadał rozmiarowi starej (rozmiar określa specjalny parametr). Wyżej opisaną metodą wybierane są dwa osobniki z poprzedniej populacji (nie może zostać wybrany dwa razy ten sam) i zachodzi proces krzyżowania. Rekombinacja w przypadku tego problemu nie może zachodzić całkowicie losowo, ponieważ bardzo łatwo uzyskać rozwiązanie niedopuszczalne. W celu przeprowadzenia rekombinacji, która pozwoli na wymianę fragmentów rozwiązania pomiędzy dopasowaniami i w miarę prosty mechanizm naprawczy aplikacja wyszukuje najpierw najdłuższy wspólny podciąg między rodzicami i skupia się na pozostałych fragmentach dopasowania (a więc tylko na miejscach, które różnią się między sekwencjami). Do wyznaczenia najdłuższego wspólnego podciągu dwóch rodziców wykorzystano klasyczny algorytm LCSb (Longest Common Subsequence; przypis: *“Dynamic Programming: LCS”* - Er. Kamaljeet Kaur, Er. Neeti Taneja, DOI:10.23956/ijarcsse/V7I6/0131) oparty na programowaniu dynamicznym. Został on zmodyfikowany w taki sposób, aby oprócz samego wspólnego podciągu zwracał również pozycje jego kolejnych elementów w obu analizowanych chromosomach, co pozwala później

wyznaczyć fragmenty pomiędzy nimi ("luki"). Elementy należące do najdłuższego wspólnego podciągu przepisywane są na swoje miejsca do dziecka bez zmian. Dla każdej "luki" natomiast losowo wybierany jest odpowiadający jej fragment jednego z rodziców i przepisywany jest do potomka. Analogicznie obsługiwane są fragmenty znajdujące się przed pierwszym i za ostatnim elementem wspólnego podciągu. W efekcie powstaje chromosom zawierający wspólne elementy obu rodziców oraz losowo wybrane fragmenty pochodzące od jednego z nich w miejscach różnic.

W następnym kroku rozpoczyna się mechanizm naprawczy. Dla każdej sekwencji instancji tworzony jest licznik określający stopień jej pokrycia przez aktualnie analizowany chromosom (mechanizm liczników bardzo podobny do tego, wykorzystywanego przy generowaniu populacji początkowej). Chromosom przeglądany jest od początku do końca. Jeżeli dany nukleotyd pozwala przesunąć przynajmniej jeden licznik, jest on uznawany za "użyteczny" i pozostaje w rozwiązaniu. Jeżeli nie, sprawdzane jest, czy jego usunięcie nie umożliwiłoby natychmiastowego wykorzystania kolejnego nukleotydu. Jeżeli tak, znak zostaje uznany za nadmiarowy i usunięty. W przeciwnym przypadku do chromosomu wstawiany jest dodatkowy nukleotyd pochodzący z jednej (losowo wybranej) sekwencji, która nie została jeszcze w pełni pokryta i ponownie sprawdzane jest czy problematyczny nukleotyd teraz już będzie użyteczny. Jeżeli nie, wstawiany jest kolejny nukleotyd z losowo wybranej sekwencji z instancji i ta procedura będzie się powtarzać tak długo aż nukleotyd nie zostanie uznany za użyteczny lub nie będzie możliwe jego usunięcie w przypadku stwierdzenia, że kolejny jest użytecznym. Taki mechanizm pozwala na dość częste usuwanie elementów z środka dopasowania, co może zwiększać prawdopodobieństwo stworzenia rozwiązania wciąż niedopuszczalnego i w rezultacie wydłuża czas trwania obliczeń jednej iteracji. Jest to jednak jedna z najszybszych metod na skrócenie dopasowania i często może się zdarzać, że nukleotydy, które usunęła faktycznie były nadmiarowe, a rozwiązanie po ich usunięciu udało się doprowadzić do formy dopuszczalnego. Kolejnym mechanizmem, który może skrócić uzyskane dopasowanie jest sprawdzanie liczników po każdym elemencie. W momencie, w którym liczniki dla wszystkich sekwencji z instancji osiągną wartości ich długości, czyli instancja zostanie pokryta, reszta chromosomu zostanie automatycznie uznana za nadmiarową i odrzucona, bez względu na jej pozostałą długość.

Po zakończeniu pierwszego mechanizmu naprawczego (opierającego się o przegląd chromosomu dziecka od początku do końca) wykonywana jest końcowa weryfikacja poprawności. Ponownie wyznaczany jest stopień pokrycia wszystkich sekwencji instancji (poprzez mechanizm liczników). Jeżeli którakolwiek z sekwencji nie została jeszcze w pełni pokryta, uruchamiana jest procedura rozszerzania potomka. Polega ona na dodawaniu na końcu chromosomu kolejnych nukleotydów pochodzących z losowo wybranych niepokrytych jeszcze sekwencji. Maksymalna liczba takich prób dodania nukleotydu jest określona przez specjalny parametr algorytmu. Jeżeli w tym limicie nie uda się uzyskać rozwiązania dopuszczalnego, potomek jest odrzucany i nie zostaje dodany do nowej populacji. W przeciwnym przypadku zwracany jest gotowy chromosom potomka, który może uczestniczyć w dalszych etapach działania algorytmu genetycznego. Rozwiązanie z limitem prób wydłużenia potomka wybrano w celu uniknięcia tworzenia potomków o naprawdę słabej funkcji celu. Bez względu na to, czy potomka udało się czy nie udało dodać do nowej populacji, jeżeli jej rozmiar nie odpowiada jeszcze wartości opisywanej przez specjalny parametr, kolejne dwa osobniki są losowane (mogą wypaść te same) i cały proces reprodukcji rozpoczyna się od nowa.

e) Mutacje

Gdy uda się już uzyskać wszystkie osobniki nowej populacji, dodawane są mutacje. Parametr określający ich ilość operuje na wartości procentowej (jakie jest prawdopodobieństwo mutacji dla konkretnej pozycji w konkretnej sekwencji), dlatego faktyczna ilość mutacji zależy bezpośrednio od rozmiaru populacji oraz długości chromosomów aktualnych osobników. Każda mutacja wykonywana jest niezależnie od pozostałych.

W pierwszym kroku losowany jest osobnik, który zostanie poddany mutacji, oraz pozycja w jego chromosomie. Następnie, wykorzystując ten sam mechanizm liczników, który stosowany jest podczas weryfikacji dopuszczalności rozwiązań, wyznaczany jest stopień pokrycia wszystkich sekwencji instancji przez fragment chromosomu poprzedzający wylosowaną pozycję. Na tej podstawie określone są sekwencje, które nie zostały jeszcze w pełni pokryte. Jeżeli istnieją jeszcze niepokryte sekwencje, losowana jest jedna z nich. Następnie sprawdzane jest, jaki nukleotyd powinien zostać aktualnie dopasowany w tej sekwencji zgodnie ze stanem liczników. Jeżeli odpowiada on już znakowi znajdującemu się w wybranej pozycji chromosomu lub wskazana sekwencja została jednak całkowicie pokryta, mutacja zostaje odrzucona i ponawiana jest kolejna próba. W przeciwnym przypadku w wybranym miejscu chromosomu wstawiany jest dodatkowy nukleotyd pochodzący z wylosowanej sekwencji instancji. Tak zdefiniowana mutacja polega więc na kontrolowanym wstawieniu nowego nukleotydu, który może przyczynić się do lepszego pokrycia sekwencji instancji. Dzięki temu każda zaakceptowana mutacja nie prowadzi do powstania rozwiązania niedopuszczalnego i nie trzeba stosować dodatkowego mechanizmu naprawczego. Po wykonaniu mutacji uruchamiana jest dodatkowa procedura upraszczania zmodyfikowanego chromosomu. Jej działanie opiera się na ponownym przejściu przez całe rozwiązanie przy wykorzystaniu mechanizmu liczników określających stopień pokrycia poszczególnych sekwencji instancji. W trakcie przechodzenia przez chromosom, usuwane są nieużywane nukleotydy ("nieużyteczne" - nie zmieniające wartości ani jednego licznika) i nadmiarowa część chromosomu z końca, jeżeli wszystkie liczniki osiągną wartości równe długości sekwencji z instancji wcześniej. Pozwala to ograniczyć jego długość i poprawić wartość funkcji celu bez ryzyka utraty dopuszczalności rozwiązania.

f) Sprawdzenie najlepszego osobnika i aktualizacja interfejsu

Zarówno po zakończeniu operacji reprodukcji, jak i mechanizmu mutacji, sprawdzane jest, czy w nowej populacji pojawiło się rozwiązanie lepsze od najlepszego znalezione dotychczas. Jeżeli długość chromosomu któregoś z osobników jest mniejsza od aktualnej wartości funkcji celu najlepszego rozwiązania, zapamiętana zostaje nowa wartość funkcji celu, a znaleziony osobnik dodany zostaje do uprzednio wyczyszczonej listy najlepszych wyników. Jeżeli długość chromosomu jest taka sama jak aktualna najlepsza wartość funkcji celu, rozwiązanie również zostaje zapisane na liście, o ile nie znajdowało się tam jeszcze. Dzięki temu algorytm przechowuje wszystkie różne rozwiązania o najlepszej znalezionej jakości.

Po zakończeniu każdej iteracji najważniejsze informacje o aktualnym stanie algorytmu są przekazywane do wątku odpowiedzialnego za obsługę interfejsu użytkownika. Aktualizowana jest zawartość pól tekstowych prezentujących bieżącą populację, listę najlepszych znalezionych rozwiązań oraz wartość funkcji celu najlepszego osobnika (lub najlepszych osobników), a także czas działania algorytmu i procent wykonania obliczeń (na podstawie ilości pozostałych iteracji). Dodatkowo na podstawie pierwszego z najlepszych

rozwiązań generowana jest graficzna reprezentacja dopasowania sekwencji oraz aktualizowany jest wykres przedstawiający zmianę wartości funkcji celu w kolejnych iteracjach.

3. Opis interfejsu aplikacji okienkowej

Interfejs aplikacji składa się z jednego, sporego okna “Algorytm genetyczny dla problemu minimalnego bezbłędneho dopasowania sekwencji”. Okazjonalnie wyświetlane są również okna typu MessageBox z pojedynczymi komunikatami. W interfejsie można wyróżnić 6 sekcji: generator instancji, instancja, parametry metaheurystyki, metaheurystyka, wyniki, oraz wykres zmian funkcji celu w czasie działania algorytmu.

Obraz 3.1 - początkowy widok interfejsu (po włączeniu).

Część elementów okna, np. wykres czy wyniki pojawiają się dopiero po wykonaniu odpowiedniej interakcji np. po uruchomieniu specjalnym przyciskiem algorytmu genetycznego. Wszystkie elementy interfejsu są dobrze chronione, np. przed wpisywaniem do pól tekstowych niepoprawnych wartości lub klikaniem przycisków w niepoprawnej kolejności (a nawet próbami uruchomienia dwóch algorytmów czy testów na raz).

Generator instancji i instancja

Na generator instancji składają się 4 pola do wpisywania wartości parametrów generatora oraz przyciski “generuj instancję” oraz “sprawdź instancję”. Jeżeli skorzysta się z opcji generowania instancji bez podania wartości parametrów, zostaną one wybrane losowo (opisane szczegółowo w punkcie 2a). Pod spodem znajduje się podgląd wygenerowanej lub wpisanej instancji, który w każdym momencie można edytować (spowoduje to jednak zablokowanie przycisku “uruchom algorytm genetyczny” i potrzebne będzie skorzystanie z przycisku “sprawdź instancję”). Jeżeli instancja była generowana wyświetlane jest również dopasowanie optymalne, na którym bazował generator przy tworzeniu instancji, a także jego długość.

Obraz 3.2 - Generator instancji i instancja.

Parametry metaheurystyki i metaheurystyka

W części związanej z parametrami metaheurystyki znajdują się pola tekstowe do wpisywania ich wartości a także opcja automatycznej zmiany wartości prawdopodobieństwa występowania mutacji w przypadku przestoju (braku zmiany wartości funkcji celu w kolejnych iteracjach). W przypadku pozostawienia któregoś pola pustego, jako wartość parametru zostanie wybrana wartość domyślna.

Część związana z metaheurystyką składa się z 5 przycisków lub 6 (ale tylko dwóch aktywnych) w trakcie działania metaheurystyki. Cztery pierwsze przyciski dotyczą testowego uruchamiania algorytmu genetycznego - oznacza to generowanie na podstawie parametrów generatora instancji 10 różnych instancji (o tych samych parametrach; jeżeli nie wybrano konkretnych wartości, ich losowanie zajdzie tylko raz) oraz uruchamianie dla każdej z nich algorytmu genetycznego. Działanie testu nie jest w jakiegokolwiek sposób raportowane w interfejsie, wyniki testów zapisują się do plików w formacie CSV. W jednym z nich zapisywane są wyniki dla każdej instancji z osobna, w drugim średnie wartości funkcji celu i czasu trwania obliczeń dla wszystkich 10 uruchomień algorytmów. Obliczenia są zrównoleglone, w trakcie trwania testu przyciski interfejsu są zablokowane. Na zakończenie testu wyświetlana jest informacja o ścieżce, pod którą zapisane zostały wyniki. W interfejsie są też opcje zapisu wygenerowanych w ramach testu 10 instancji do pliku w prostym formacie JSON oraz wczytania ich z powrotem.

Największy przycisk w części "Metaheurystyka" to uruchomienie algorytmu genetycznego w normalnym trybie - z podglądem funkcji celu, aktualnej populacji, czy czasu działania. Po kliknięciu go, w jego miejscu pojawiają się przyciski "Stop" oraz "Pauza" / "Wznów", pozwalające na całkowite przerwanie działania metaheurystyki lub jej tymczasowe zatrzymanie.

Parametry metaheurystyki

Rozmiar populacji (domyślnie 100):

Ilość iteracji algorytmu (domyślnie 200):

Prawdopodobieństwo wystąpienia mutacji (domyślnie 0,0003):

Procent osobników pozostawianych w populacji (domyślnie 0,25):

Presja reprodukcyjna (premiowanie lepszych osobników; domyślnie 15,0):

Liczba prób wydłużania nowego osobnika (domyślnie 2):

Czy automatyczna zmiana wartości prawdopodobieństwa mutacji na większą, gdy przestój? ☒

Tymczasowa wartość (domyślnie 0,004):

Długość przestoju (domyślnie 20):

Metaheurystyka

Utwórz instancje testowe (10 instancji) | Uruchom test (na 10 instancjach testowych)

Zapisz instancje testowe do pliku | Wczytaj instancje testowe z pliku

Uruchom algorytm genetyczny

Metaheurystyka

Utwórz instancje testowe (10 instancji) | Uruchom test (na 10 instancjach testowych)

Zapisz instancje testowe do pliku | Wczytaj instancje testowe z pliku

Stop | **Wznów**

Postęp: 3%

Obraz 3.3 - Parametry metaheurystyki i metaheurystyka przed oraz po uruchomieniu algorytmu.

Wyniki oraz wykres zmian funkcji celu w czasie działania algorytmu.

Po uruchomieniu algorytmu genetycznego poza procentową wartością opisującą jego postęp (dokładniej: ile % iteracji już upłynęło), widoczna w interfejsie jest również aktualna populacja (co pozwala śledzić reprodukcje i mutacje na bieżąco), czas działania metaheurystyki w sekundach, aktualna najlepsza wartość funkcji celu wraz ze wszystkimi rozwiązaniami (osobnikami), które ją osiągnęły (rozwiązania można przewijać strzałkami w polu tekstowym) oraz dopasowanie wszystkich sekwencji z instancji do pierwszego rozwiązania z aktualnej listy najlepszych (w pierwszej linijce znajduje się dopasowanie, a w kolejnych kolejne sekwencje z instancji).

Wyniki

Aktualna populacja:

```

ACCGCGATGTGGTGAACACATATCTCTCGTTACTCTATC
ACCGCGAGTGGGTAGACACTCATCTACTCAGTTTCATC
ACCGCGATGTGGTAGACTCACATATCCGTTCTATAC
ACCGCGAGATGGAGTGACTACATCTCTCGTTCACTCGTT
ACCGCGAGGTGATGGTGACTCATCTCTGTACTTACGTTG
ACCGCGATGGTGGTCACTACTCATCTCTCGTTTCTATCC
ACCGCGATGGTGGTGAACATCATCTCTCGTTTACTAGT
ACCGCGATGTGATGGTACACTCATCTCTGACCGTTACTA
ACCGCGATGGTGGTGAACATCATCTCTCGTTTACTAGT
ACCGCGAGTGGTGGATCAACTACATCTCTCTCGTTACGT
ACCGCGAGTGGTGGACTAACTCATATCTCCGTTACTCTATA

```

Aktualna najlepsza wartość funkcji celu:

Aktualne najlepsze rozwiązanie:

ACCGCGATGTGGTGAACATCATCTCTCTCGTTTATACAGT

```

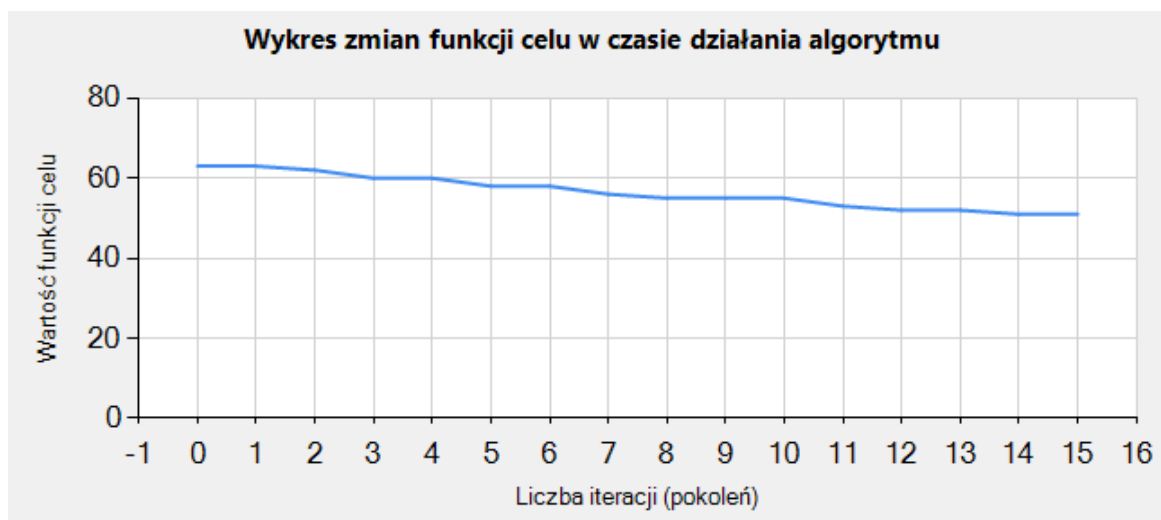
ACCGCGATGTGGTGAACATCATCTCTCGTTTATACAGTAGTA
A GCG TG G AA TC TC CT T TTTA A G G A
ACCG TGA A TCA ATC C GTT A CAG G
ACC CG G T AACT AT TC C C TT A A GTA A
AC GCG TG G A T T AT GTTTA A A AGTA
ACC CGA G GG G T TCAT TC GTTTA CA
AC A GAT CT A C C C GTTTA CAG G A
AC G G TG G TGA C C T ATC CT A CAG A
AC G GA G G T A T TC CT T AT CAGTA
C G G G T T AA AT A C C GTT ATA G G A
C GCG TGA A TCAT TC C GTTTA G GT
AC GCG G G A CT TC C GTT A AC GTA
C A G G TGA CTAT TC C C TTTATA AG A

```

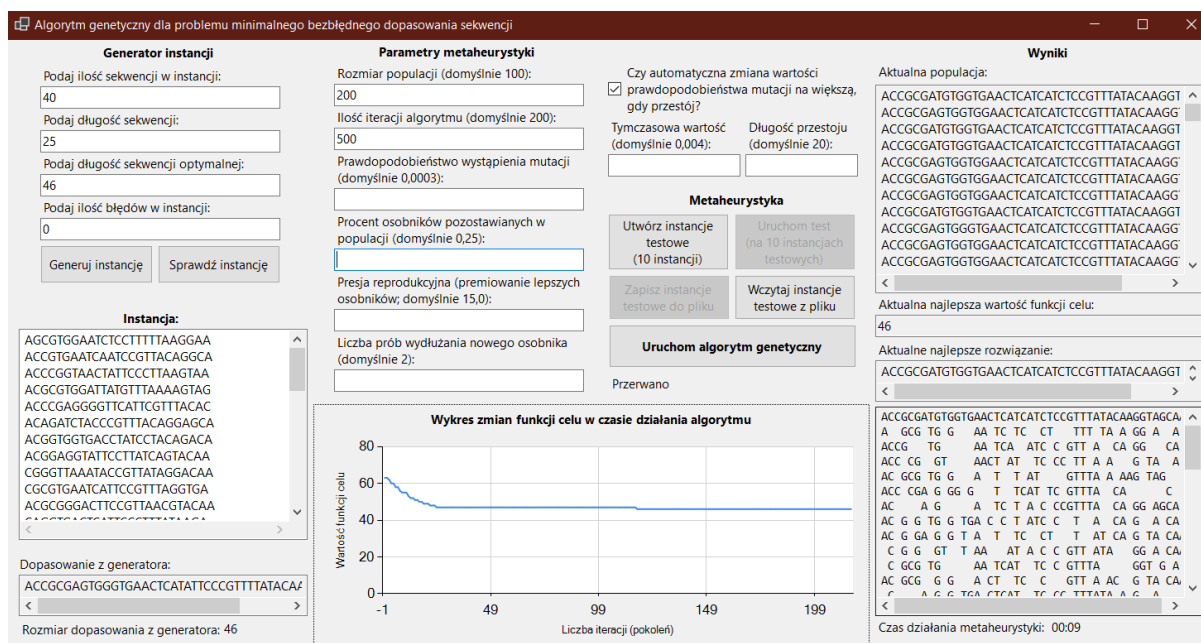
Czas działania metaheurystyki: 00:02

Obraz 3.4 - Informacje wyświetlane w trakcie i po zakończeniu działania metaheurystyki w części wyników.

Po uruchomieniu algorytmu genetycznego w interfejsie pojawia się również wykres zmian funkcji celu w czasie działania algorytmu, uzupełniany na bieżąco co iterację. Pokazuje on dynamikę spadków wartości funkcji celu oraz jej przestoje.



Obraz 3.5 - Wykres zmian funkcji celu w czasie działania algorytmu.



Obraz 3.6 - Interfejs po zakończeniu działania algorytmu.

4. Opis procesu strojenia parametrów

W tej części sprawozdania znajdują się testy wpływu różnych wartości parametrów: prawdopodobieństwo wystąpienia mutacji, procent osobników pozostawionych w populacji, presja reprodukcyjna oraz liczba prób wydłużania nowego osobnika na funkcję celu. Testy były przeprowadzane na 10 instancjach wygenerowanych na podstawie tych samych parametrów generatora (długość sekwencji w instancji, ilość sekwencji w instancji, długość sekwencji optymalnej, ilość błędów w instancji), a w tabelach zamieszczono średnie wartości funkcji celu oraz czasu działania dla wszystkich 10 instancji przy konkretnych wartościach parametrów algorytmu genetycznego.

Przeprowadzone zostały również testy wpływu zmian wartości parametrów na czas trwania obliczeń, na tych samych instancjach co wpływ na funkcję celu.

Zawarte w tej części testy były wykonywane na instancjach o parametrach:

liczba sekwencji w instancji = 30,

długość sekwencji w instancji = 80,

długość sekwencji optymalnej z generatora = 140,

liczba błędów w instancji = 0.

Są to wartości dobrane pod kątem stworzenia instancji dość trudnych, dla których nie da się w krótkim czasie (przy małej ilości iteracji i niewielkim rozmiarze populacji) znaleźć rozwiązania optymalnego. Dzięki temu, można zaobserwować bezpośredni wpływ zmian wartości parametrów na jakość dopasowania.

Test pierwszy - presja reprodukcyjna

Parametry:

rozmiar populacji = 50,

ilość iteracji algorytmu = 50,

prawdopodobieństwo wystąpienia mutacji = 0,01,

procent osobników pozostawianych w populacji = 0,2,

presja reprodukcyjna - strojony,

liczba prób wydłużania nowego osobnika = 5.

Presja reprodukcyjna	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
2	211,1	21,3634
3	209	24,1486
4	208,9	22,4303
5	207,9	24,2108
6	207,2	23,1077
7	205,8	23,1132
8	207,1	23,0903
9	206,5	23,7271
10	204,2	23,3412
11	205,6	22,2419
12	204,8	23,6973
13	204,2	23,2974
14	205,8	25,0207
15	203	23,7456
16	203,3	25,437
17	204,8	25,8025
18	204,2	25,8172
19	203,7	24,8843
20	203,5	25,2258

21	203,7	22,5726
22	203,7	24,6894

Tabela 4.1 - wyniki testów parametru "presja reprodukcyjna".

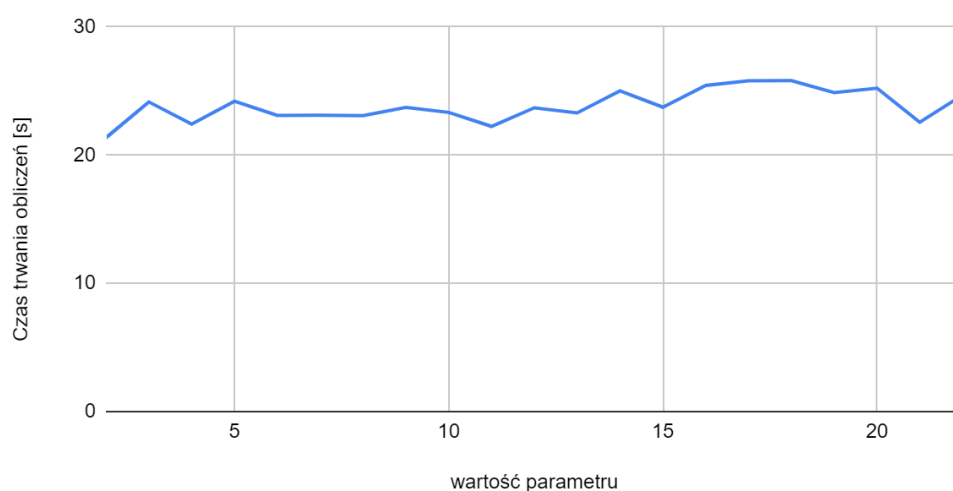
Wykres zależności funkcji celu od zmian wartości parametru "presja reprodukcyjna"



Wykres 4.1 - zależność funkcji celu od zmian wartości parametru "presja reprodukcyjna" - na podstawie Tabeli 4.1.

Najlepszy wynik uzyskała wartość parametru presja reprodukcyjna 15. Początkowo widać spadki wartości funkcji celu przy coraz większych wartościach tego parametru, następnie powyżej wartości 15 wpływ na funkcję celu zmniejsza się i funkcja celu pozostaje na podobnym poziomie bez względu na wartość tego parametru. Z tego względu wartość 15 wydaje się najlepsza i będzie ona wykorzystywana jako wartość domyślna w kolejnych testach.

Wykres zależności czasu trwania obliczeń metaheurystyki od zmian wartości parametru "presja reprodukcyjna"



Wykres 4.2 - zależność czasu trwania obliczeń metaheurystyki od zmian wartości parametru "presja reprodukcyjna" - na podstawie Tabeli 4.1.

Zmiany wartości parametru “presja reprodukcyjna” nie mają znaczącego wpływu na czas trwania obliczeń, co jest uzasadnione tym, że parametr ten nie wpływa w żaden sposób na złożoność obliczeniową kodu, a jedynie wartości prawdopodobieństwa selekcji danego osobnika do przeżycia/reprodukcji.

Test drugi - procent osobników pozostawianych w populacji

Parametry:

rozmiar populacji = 100,

ilość iteracji algorytmu = 200,

prawdopodobieństwo wystąpienia mutacji = 0,01,

procent osobników pozostawianych w populacji - strojony

presja reprodukcyjna = 15,

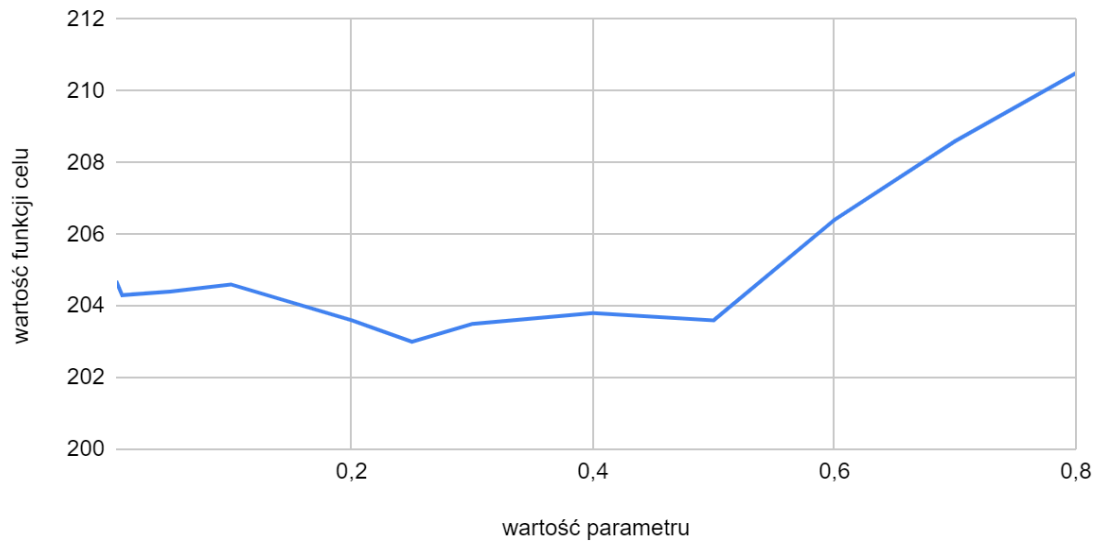
liczba prób wydłużania nowego osobnika = 5.

Procent osobników pozostawianych w populacji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
0,005	204,7	30,0516
0,01	204,3	28,6083
0,05	204,4	27,2956
0,1	204,6	26,5713
0,2	203,6	23,6353
0,25	203	21,1986
0,3	203,5	19,7044
0,4	203,8	17,2223
0,5	203,6	13,1078
0,6	206,4	10,6342

Tabela 4.2 - wyniki testów parametru “procent osobników pozostawianych w populacji”.

Najlepsze wyniki uzyskały wartości parametru “procent osobników pozostawianych w populacji” 0,2 i 0,3, stąd sprawdzona dodatkowo wartość 0,25, która faktycznie okazała się najlepszą i będzie wykorzystywana jako wartość domyślna w kolejnych testach.

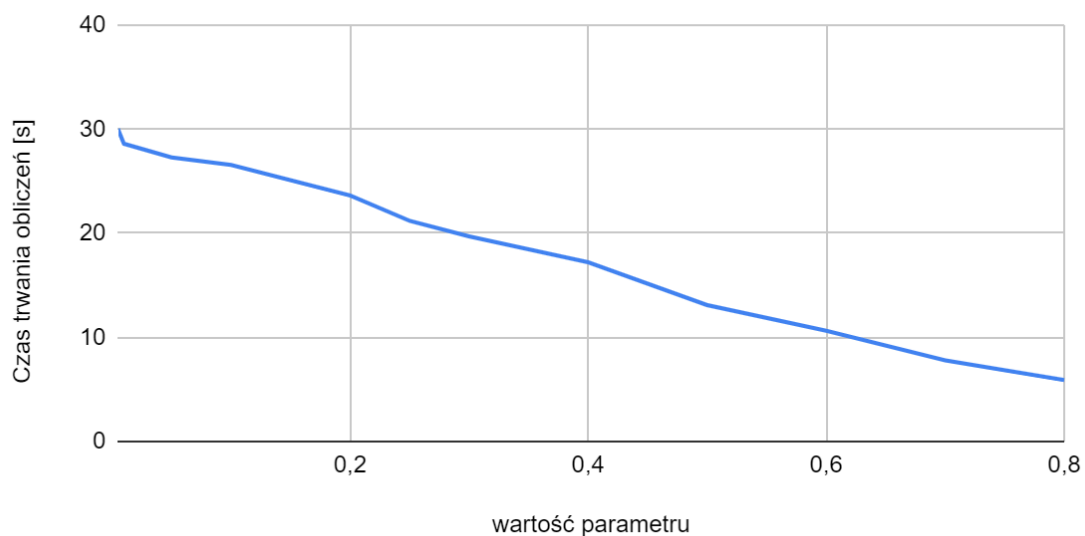
Wykres zależności funkcji celu od zmian wartości parametru "Procent osobników pozostawianych w populacji"



Wykres 4.3 - zależność funkcji celu od zmian wartości parametru "procent osobników pozostawianych w populacji" - na podstawie Tabeli 4.2.

Procent osobników pozostawianych w populacji ma bardzo mały wpływ na wartość funkcji celu tak długo, jak jest mniejszy od wartości 0,5 - jeżeli jest od niej większy wartość funkcji celu zaczyna drastycznie rosnąć, ponieważ mechanizm reprodukcji staje się coraz rzadszy, a jest on ważnym mechanizmem tworzenia osobników, również tych o mniejszej wartości funkcji celu.

Wykres zależności czasu trwania obliczeń metaheurystyki od wartości parametru "Procent osobników pozostawianych w populacji"



Wykres 4.4 - zależność czasu trwania obliczeń metaheurystyki od zmian wartości parametru "procent osobników pozostawianych w populacji" - na podstawie Tabeli 4.2.

Im większa wartość parametru tym krócej działa algorytm - wynika to z mniejszej ilości wywołań funkcji odpowiedzialnej za reprodukcję, której liczba wywołań ma duży wpływ na długość czasu trwania metaheurystyki. Wartość 0,25 wydaje się dobrym wyborem, ponieważ skraca czas działania algorytmu (w porównaniu do mniejszych wartości), ale nie powoduje jeszcze wzrostu wartości funkcji celu, tak jak większe wartości.

Test trzeci - liczba prób wydłużania nowego osobnika

Parametry:

rozmiar populacji = 50,

ilość iteracji algorytmu = 50,

prawdopodobieństwo wystąpienia mutacji = 0,01,

procent osobników pozostawianych w populacji = 0,25,

presja reprodukcyjna = 15,

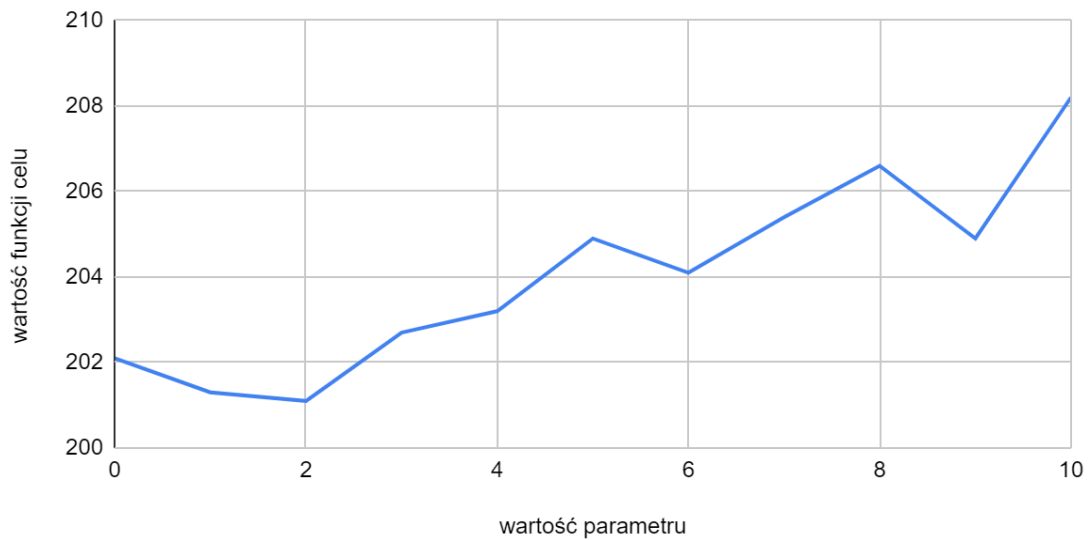
liczba prób wydłużania nowego osobnika - strojony.

Liczba prób wydłużania nowego osobnika	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
0	202,1	48,869
1	201,3	51,6541
2	201,1	40,6118
3	202,7	29,5542
4	203,2	26,4512
5	204,9	22,4291
6	204,1	18,6577
7	205,4	16,0253
8	206,6	14,7309
9	204,9	13,2064
10	208,2	11,3593

Tabela 4.3 - wyniki testów parametru "liczba prób wydłużania nowego osobnika".

Najlepszy wynik uzyskała wartość parametru "liczba prób wydłużania nowego osobnika" 2 i zostanie ona wykorzystana jako wartość domyślna w kolejnych testach.

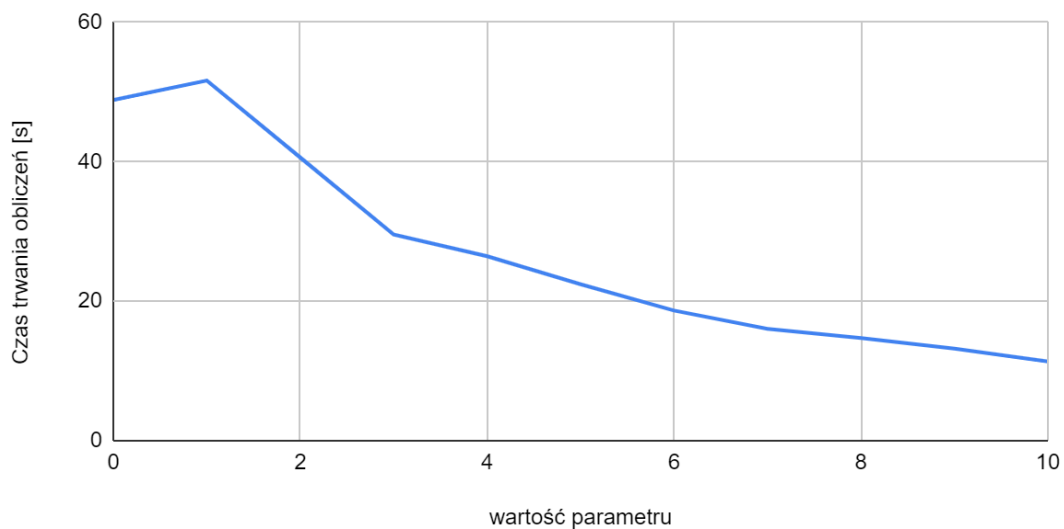
Wykres zależności funkcji celu od zmian wartości parametru "Liczba prób wydłużania nowego osobnika"



Wykres 4.5 - zależność funkcji celu od zmian wartości parametru "liczba prób wydłużania nowego osobnika" - na podstawie Tabeli 4.3.

Większe wartości tego parametru powodują gorsze wyniki funkcji celu, ponieważ przechowywanie sztucznie wydłużonych w celu zapewnienia dopuszczalności osobników nie jest najkorzystniejsze dla algorytmu genetycznego i lepiej jest takie osobniki odrzucić i spróbować na drodze reprodukcji skonstruować nowe, krótsze. Mimo to, niewielkie wydłużenie, np. o 2 może pomóc uzyskać osobniki o nowych kombinacjach różnych fragmentów dopasowań, co przełoży się na lepsze wyniki funkcji celu po kolejnych iteracjach.

Wykres zależności czasu trwania obliczeń metaheurystyki od wartości parametru "liczba prób wydłużania nowego osobnika"



Wykres 4.6 - zależność czasu trwania obliczeń metaheurystyki od zmian wartości parametru "liczba prób wydłużania nowego osobnika" - na podstawie Tabeli 4.3.

Wraz z większymi wartościami parametru spada czas trwania obliczeń metaheurystyki (i to dość znacząco) - wynika to z tego, że więcej prób wydłużania nowego osobnika gwarantuje, że częściej osobnik uzyskany na drodze krzyżowania będzie miał dopuszczalny chromosom i będzie mógł zostać dodany do nowej populacji, a to z kolei oznacza mniej wywołań funkcji odpowiedzialnej za mechanizm reprodukcji. W sytuacji, gdy pożądanym jest krótki czas działania metaheurystyki, można rozważyć również większe wartości parametru, takie jak 3 lub 7.

Test czwarty - prawdopodobieństwo wystąpienia mutacji

Wartości parametrów:

rozmiar populacji = 50,

ilość iteracji algorytmu = 50,

prawdopodobieństwo wystąpienia mutacji - strojony,

procent osobników pozostawianych w populacji = 0,25,

presja reprodukcyjna = 15,

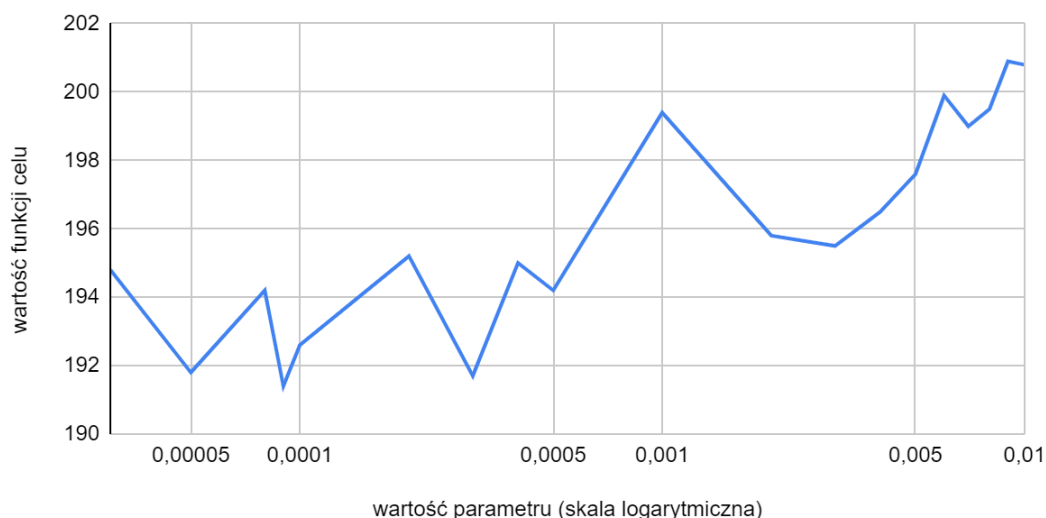
liczba prób wydłużania nowego osobnika = 2.

Prawdopodobieństwo wystąpienia mutacji	Średnia wartość funkcji celu
0,00003	194,8
0,00005	191,8
0,00008	194,2
0,0001	192,6
0,0002	195,2
0,0003	191,7
0,0004	195
0,0005	194,2
0,001	199,4
0,002	195,8
0,003	195,5
0,004	196,5
0,005	197,6
0,006	199,9
0,007	199
0,008	199,5
0,009	200,9
0,01	200,8

Tabela 4.4 - wyniki testów parametru "prawdopodobieństwo wystąpienia mutacji".

Najlepszy wynik uzyskała wartość parametru "prawdopodobieństwo wystąpienia mutacji" 0,0003.

Wykres zależności funkcji celu od wartości parametru "prawdopodobieństwo wystąpienia mutacji"



Wykres 4.7 - zależność funkcji celu od zmian wartości parametru "prawdopodobieństwo wystąpienia mutacji" - na podstawie Tabeli 4.4.

Większa ilość mutacji przy wartościach parametrów liczba iteracji 50 oraz rozmiar populacji 50 powoduje wzrost wartości funkcji celu, jednak nie jest on stabilny. Wyższe prawdopodobieństwo mutacji przekłada się wprost na większą losowość algorytmu genetycznego, co znaczy, że wyniki będą mniej powtarzalne i ciężiej zaobserwować pewną zależność funkcji celu od wartości parametru - czasem większa ilość mutacji przełoży się na lepszy wynik, a w innym uruchomieniu metaheurystyki na gorszy. Mniejsze wartości parametru związanego z mutacjami skutkują bardziej przewidywalnymi (stabilnymi) wynikami, a często również lepszymi, co jest zdecydowanie bardziej pożądaną cechą.

Wykres zależności czasu trwania obliczeń metaheurystyki od wartości parametru "prawdopodobieństwo wystąpienia mutacji"



Wykres 4.8 - zależność czasu trwania obliczeń metaheurystyki od zmian wartości parametru "prawdopodobieństwo wystąpienia mutacji" - na podstawie Tabeli 4.4.

Wartość prawdopodobieństwa wystąpienia mutacji wpływa na czas trwania obliczeń, zwiększając go, jednak ten efekt jest widoczny dopiero dla dość dużych wartości tego parametru (powyżej 0,005).

Test piąty - wychodzenie algorytmu z optimum lokalnego

Takie wartości parametrów mocno wspierają intensyfikację i ograniczają dywersyfikację, co budzi pytanie czy przy dłuższym czasie działania algorytmu nie będzie utykał on w optimum lokalnym (być może 50 iteracji dla tak dużej instancji to zbyt mało, żeby metaheurystyka odnalazła pierwsze optimum lokalne i potrzeba opuszczenia go w celu znalezienia optimum globalnego nie występuje). W tym celu przeprowadzono serię testów dla tych samych instancji z wydłużeniem liczby iteracji (oraz większym rozmiarem populacji).

Rozmiar populacji	Liczba iteracji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
100	200	185,6	119,7466
100	300	182,4	148,9221
100	400	177	178,3168
100	500	180,3	191,6441
100	600	179,4	232,6302
200	700	171	680,179
300	700	168,9	1102,4249
500	1000	168,9	2310,2426
1000	1000	169,9	6110,3422

Tabela 4.5 - wyniki testów wychodzenia algorytmu z optimum lokalnego.

Faktycznie wygląda to tak, jakby algorytm miał duży problem z przekroczeniem wartości 169, pytanie tylko czy jest to winą zbyt słabej dywersyfikacji czy trudnych (dużych) instancji. W tym celu, sprawdzane są również różne wartości parametrów rozmiar populacji i liczba iteracji dla większych wartości parametru "Prawdopodobieństwo wystąpienia mutacji", który bezpośrednio zwiększa dywersyfikację.

Rozmiar populacji	Liczba iteracji	Prawdopodobieństwo wystąpienia mutacji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
50	500	0,0003	188,5	76,3868
100	500	0,0003	180,3	191,6441
300	700	0,0003	168,9	1102,4249
50	500	0,001	188,6	80,461
100	500	0,001	183,5	191,3009
300	700	0,001	170,5	1213,7368

50	500	0,01	192	280451,5
100	500	0,01	188,9	774,7073
300	700	0,01	187	3330,5804

Tabela 4.6 - wyniki testów wychodzenia algorytmu z optimum lokalnego przy częstszych mutacjach.

Testy na różnych wartościach rozmiaru populacji i liczby iteracji, pokazują że trudno wprost zauważyć poprawę wyników przy zastosowaniu większej wartości prawdopodobieństwa wystąpienia mutacji - dopóki jakiegokolwiek mutacje się pojawiają spadek funkcji celu nie ustanie całkowicie, natomiast każde zwiększenie ich liczby powoduje gorsze wyniki w określonej liczbie iteracji, ponieważ dużo więcej czasu programowi zajmuje osiągnięcie jakiegokolwiek lokalnego minimum. Na konkretnym przykładzie instancji bez błędów (30 sekwencji o długości 80) o sekwencji optymalnej długości 140:

AGGCGAGCAGTCGGTTACAGAAAATCCATGGATTCTCACTTTTCGTCATAACATCGACATGAAGCTTTCCTCCAGATAACTAGGGCGAACGTATTCTTCTCATGGAAGAA
GACGGAACTCCAGCACTACGGTAGGAATT

Przy wartościach parametrów:

rozmiar populacji = 100,

ilość iteracji algorytmu = 500,

prawdopodobieństwo wystąpienia mutacji = 0,0003,

procent osobników pozostawianych w populacji = 0,25,

presja reprodukcyjna = 15,

liczba prób wydłużania nowego osobnika = 2.

Najlepsze znalezione dopasowanie ma długość 174:

AGCGACAGTCAGTCGTACAGATACAGTACGATCGATGTCATCTATCTCTATCTGATACTATGACGACATCGACATGATCTCTCAGTCTCAGCTAGCTAGCTAGCAGCATA
CGTATCTACTCTCGATAGAGACGACGATGACGATCAGCATGCACGACATGACGTAGGAATT

A wykres zmian funkcji celu wygląda tak:



Obraz 4.1 - Wykres zmian funkcji celu w czasie działania algorytmu dla prawdopodobieństwa wystąpienia mutacji 0,0003 i liczby iteracji 500 - z interfejsu programu.

Wyraźnie widoczne (po wyświetlonej w każdej iteracji aktualnej populacji) jest zjawisko utraty różnorodności - mała liczba mutacji powoduje że wszystkie osobniki stają się bardzo podobne i różnice między nimi stają się wynikiem przede wszystkim bieżących mutacji. Na wykresie zmian funkcji celu widać, że choć początkowo jej wartość spada bardzo szybko, potem spadki są bardzo rzadkie.

Po zmianie wartości prawdopodobieństwa wystąpienia mutacji na 0,001, najlepsze dopasowanie ma długość 177 i prezentuje się następująco (właściwie jest to jedno z dwóch, metaheurystyka znalazła dwa tej długości):

```
AGGCAGCAGTCAGTCGTAGTATCAGAATCGATCGATCTCGATCTCTATCTGATCGATCATAGCATCAGCATCAGATCATGCTCTCATGCTAGCTAGCTAGCGTACGTACT
CTGCATGCATGCTAGCTGAGCGATGCGAGTACGATGCGACGATCGACTAGCGTAGGAATT
```

A wykres zmian funkcji celu wygląda tak:



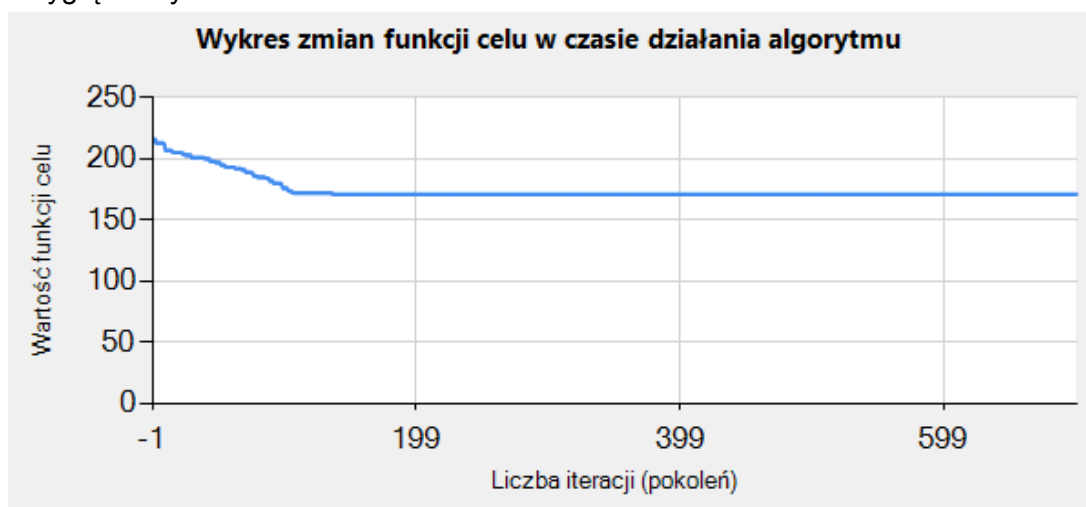
Obraz 4.2 - Wykres zmian funkcji celu w czasie działania algorytmu dla prawdopodobieństwa wystąpienia mutacji 0,001 i liczby iteracji 500 - z interfejsu programu.

Przy większej ilości mutacji wartości funkcji celu spadają w sposób stabilniejszy, jednak wolniejszy. Więcej czasu mija do momentu kiedy algorytm zacznie cechować się małą różnorodną populacją, ale spadek funkcji celu wynikający z różnorodności pierwotnych rozwiązań jest mniejszy. Mimo to, przy tej liczbie iteracji wciąż lepszy wynik osiąga mniejsza liczba mutacji.

Testując te same wartości parametru "prawdopodobieństwo wystąpienia mutacji" dla liczby iteracji równej 700, wartość 0,0003 osiągnęła wynik 171 (co ciekawe, dla równo 9 różnych dopasowań):

```
AGCGGCAGTCAGTCGTACGTACGAATCAGATCGATCTGATCATCTCATCGTCATCGTATACGTATCGATCGATGCATCGAGTCTCATCGACTCAGCTATGCAGTCGAGTAC
GATCTATCGTCTATCTCAGTAGCAGAGACGAGACGACTCGCAGCACTACGAGTAGGAATT
```

A tak wyglądał wykres zmian:



Obraz 4.3 - Wykres zmian funkcji celu w czasie działania algorytmu dla prawdopodobieństwa wystąpienia mutacji 0,0003 i liczby iteracji 700 - z interfejsu programu.

Natomiast wartość parametru "prawdopodobieństwo wystąpienia mutacji" równa 0,001, tym razem osiągnęła wynik 181:

```
AGCGAGCAGTCAGTCGTCATCAGATCGATATCGATCATGCATCTTCATCGTCATACTCGATCACTGATCGACATGACTAGATCATGACTCAGCTCAGTCGACTAGCAT
GCGAACGTATCATCTGATGCTATCGAGAGACGATGACGTAGCAGACTACGCACTGACGTACGTAGAGATT
```

Tak prezentowała się funkcja celu w kolejnych iteracjach:



Obraz 4.4 - Wykres zmian funkcji celu w czasie działania algorytmu dla prawdopodobieństwa wystąpienia mutacji 0,001 i liczby iteracji 700 - z interfejsu programu.

Gorszy wynik wartości 0,001 dla 700 iteracji w porównaniu do 500 pokazuje, że większe prawdopodobieństwo mutacji zwiększa losowość algorytmu genetycznego i różne uruchomienia na podobnych lub nawet tych samych parametrach będą mieć jeszcze mniej powtarzalne wyniki (i czasem metaheurystyka znajdzie lepsze dopasowanie w krótszym czasie, a często nie znajdzie go wcale). Te testy potwierdzają, że mimo wszystko wartość 0,0003 wydaje się dużo lepszym wyborem na uzyskanie możliwie najlepszych wyników bez względu na liczbę iteracji.

Test szósty - dynamicznie zmieniająca się wartość prawdopodobieństwa występowania mutacji

Kolejnym pomysłem na poprawę wyników jest zastosowanie dynamicznie zmieniającej się wartości prawdopodobieństwa wystąpienia mutacji. Przy mniejszych liczbach iteracji zwiększenie prawdopodobieństwa mutacji pogarsza wyniki, jednak przy większych widać, że pod koniec działania metaheurystyki zmiany funkcji celu praktycznie już nie zachodzą - stąd pomysł na dynamiczną zmianę wartości parametru związanego z mutacjami po konkretnej ilości iteracji bez zmian wartości funkcji celu. Zwiększenie ilości mutacji na krótki czas, powoduje szybsze przeszukiwanie przestrzeni rozwiązań dopuszczalnych, co może skutkować lepszymi wynikami przy większej liczbie iteracji. Jeżeli zwiększenie ilości mutacji faktycznie skutkuje zmianą funkcji celu, algorytm genetyczny wraca do oryginalnej wartości prawdopodobieństwa wystąpienia mutacji, aby dobrze przeszukać część przestrzeni rozwiązań, w której znajdują się jego nowe osobniki. Ten mechanizm został zaimplementowany jako opcjonalny i opisują go dwa parametry - liczba iteracji przestoju (bez zmian funkcji celu), po której zwiększona zostanie wartość prawdopodobieństwa wystąpienia mutacji, oraz nowa, tymczasowa wartość tego parametru. Mechanizm został

przetestowany w podobny sposób co poprzednie parametry, na tych samych bezbłędnych (0 błędów przy generowaniu instancji) 10 instancjach o wielkości 30 sekwencji o długości 80 i sekwencji optymalnej z generatora o długości 140.

Wartości parametrów:

rozmiar populacji = 100,

ilość iteracji algorytmu = 500,

prawdopodobieństwo wystąpienia mutacji (początkowe) = 0,0003,

procent osobników pozostawianych w populacji = 0,25,

presja reprodukcyjna = 15,

liczba prób wydłużania nowego osobnika = 2.

Tymczasowa wartość prawdopodobieństwa wystąpienia mutacji	Długość przestoju (liczba iteracji bez zmian funkcji celu)	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
brak (porównanie)	brak (porównanie)	181,1	207,2727
0,00075	20	181,6	217,0514
	30	181,4	202,8683
	40	181,8	206,9613
	50	178,2	207,9504
	60	180,1	214,5822
	70	179,3	221,45
	80	180,7	211,469
0,001	20	182	209,7525
	30	178,5	218,617
	40	181,1	204,0155
	50	182,6	206,3714
	60	178,6	202,8861
	70	181,5	212,2737
	80	179,3	217,4229
0,002	20	181,3	221,7115
	30	179,7	225,0755
	40	176,6	223,6325
	50	181,8	203,6993
	60	179,7	216,187
	70	178,9	205,7077
	80	178,8	205,8323
0,003	20	177,9	216,4015
	30	177,1	212,7179

	40	177,9	222,3357
	50	181	218,4893
	60	181,3	222,0613
	70	178,1	209,5016
	80	180,8	213,5698
0,004	20	174,6	239,7966
	30	179,7	239,0573
	40	180,9	222,4519
	50	177,2	248,6141
	60	177,8	220,9964
	70	178	216,2701
	80	179,1	222,8466
0,005	20	180	229,3552
	30	177,3	260,9567
	40	181	233,0401
	50	178,2	218,3561
	60	179,7	204,5827
	70	178,7	225,3301
	80	178,9	217,3926

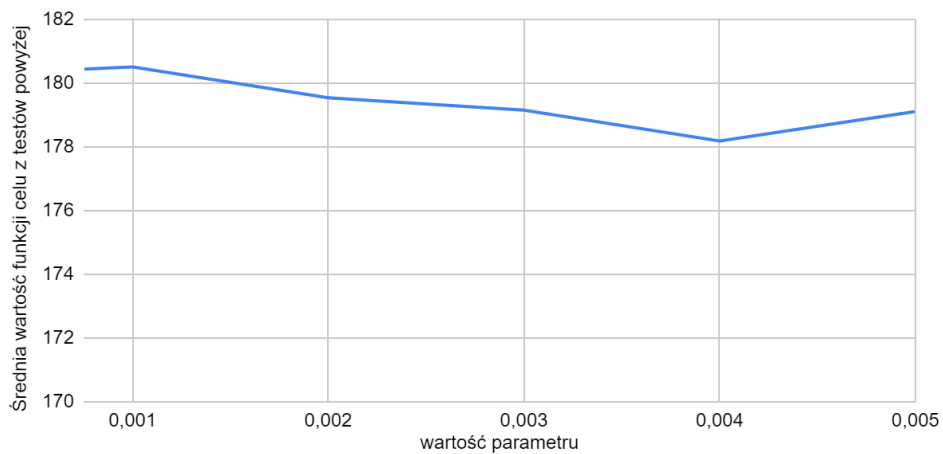
Tabela 4.7 - wyniki testów dynamicznie zmieniającej się wartości prawdopodobieństwa występowania mutacji.

Wyjątkowo dobre wyniki uzyskały wartości parametrów tymczasowa wartość prawdopodobieństwa wystąpienia mutacji - 0,004 i długość przestoju - 20. W celu sprawdzenia czy wartość ta jest wiarygodna, test został puszczony jeszcze raz i uzyskał wartość funkcji celu - 175,8 - ponownie dużo niższą niż pozostałe wyniki w tabeli.

Tymczasowa wartość prawdopodobieństwa wystąpienia mutacji	Średnia wartość ze średnich wartości funkcji celu dla różnych długości przestoju	Średni czas ze średnich czasów działania dla różnych długości przestoju
0,00075	180,4428571	211,7618
0,001	180,5142857	210,1913
0,002	179,5428571	214,5494
0,003	179,1571429	216,4395857
0,004	178,1857143	230,0047143
0,005	179,1142857	227,0019286

Tabela 4.8 - średnie wartości z testów dla tymczasowej wartości prawdopodobieństwa wystąpienia mutacji - na podstawie Tabeli 4.7.

Wykres zależności funkcji celu od wartości parametru
"tymczasowa wartość prawdopodobieństwa wystąpienia mutacji"



Wykres 4.9 - zależność funkcji celu od zmian wartości parametru "tymczasowa wartość prawdopodobieństwa wystąpienia mutacji" - na podstawie Tabeli 4.8.

Wykres zależności czasu trwania obliczeń od parametru
"tymczasowa wartość prawdopodobieństwa wystąpienia mutacji"



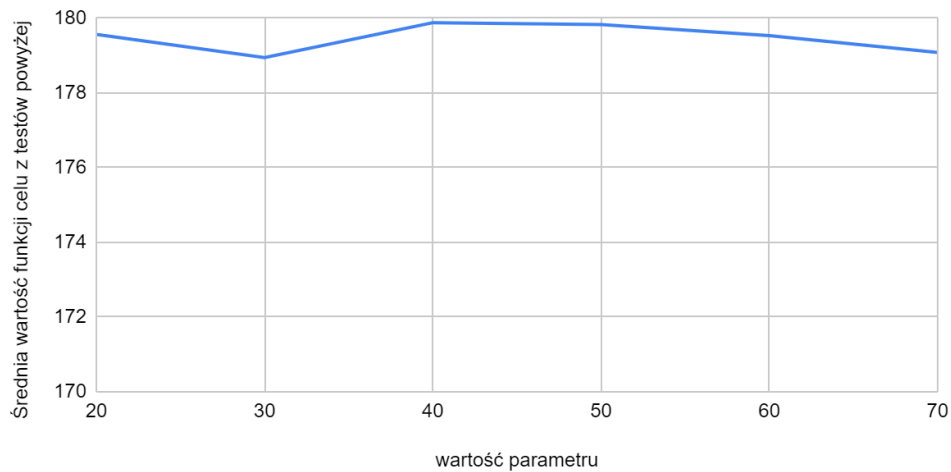
Wykres 4.10 - zależność czasu trwania obliczeń [s] metaheurystyki od zmian wartości parametru "tymczasowa wartość prawdopodobieństwa wystąpienia mutacji" - na podstawie Tabeli 4.8.

Długość przestoju	Średnia wartość ze średnich wartości funkcji celu dla różnych długości przestoju	Średni czas ze średnich czasów działania dla różnych długości przestoju
20	179,57	222,34
30	178,95	226,55
40	179,88	218,74

50	179,83	217,25
60	179,53	213,55
70	179,08	215,09
80	179,6	214,76

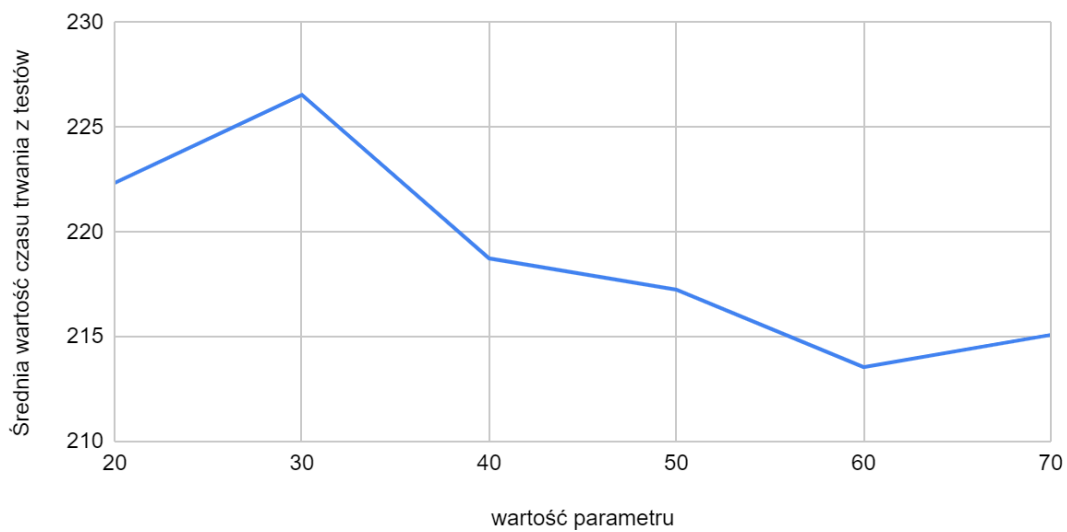
Tabela 4.9 - średnie wartości z testów dla długości przestoju - na podstawie Tabeli 4.7.

Zależność funkcji celu od wartości parametru "długość przestoju"



Wykres 4.11 - zależność funkcji celu od zmian wartości parametru "długość przestoju" - na podstawie Tabeli 4.9.

Wykres zależności czasu trwania obliczeń od wartości parametru "długość przestoju"



Wykres 4.12 - zależność czasu trwania obliczeń [s] metaheurystyki od zmian wartości parametru "długość przestoju" - na podstawie Tabeli 4.9.

Zbiórce wnioski: Wartość parametru długość przestoju ma minimalny wpływ na funkcję celu. Zauważalne jest nieznaczne skrócenie czasu trwania obliczeń przy większych wartościach parametru, co wynika z mniejszej ilości wstawianych mutacji (jest to operacja czasochłonna). Wartość parametru tymczasowa wartość prawdopodobieństwa wystąpienia mutacji osiąga najlepsze wyniki dla wartości 0,004, dla wyższych i niższych funkcja celu wzrasta. Im większe wartości przyjmuje ten parametr tym dłuższy staje się czas trwania obliczeń.

5. Testy na różnych instancjach podsumowujące jakość metaheurystyki

Stałe wartości parametrów:

prawdopodobieństwo wystąpienia mutacji = 0,0003,

procent osobników pozostawianych w populacji = 0,25,

presja reprodukcyjna = 15,

liczba prób wydłużania nowego osobnika = 2.

Jeżeli opcja z dynamicznie zmieniającą się wartością prawdopodobieństwa występowania mutacji:

Tymczasowa wartość prawdopodobieństwa wystąpienia mutacji: 0,004,

Długość przestoju (liczba iteracji bez zmian funkcji celu): 20.

W ramach każdego testu wygenerowane zostało 10 instancji spełniających określone w ramach testu warunki (wartości parametrów: ilość sekwencji, długość sekwencji, długość sekwencji optymalnej, ilość błędów) i dla różnych wartości wielkości populacji i liczby iteracji zmierzono czas trwania i końcową najlepszą funkcję celu dla każdej z nich. W tabeli poniżej zamieszczone są średnie wartości z wszystkich 10 instancji dla testowanych wartości parametrów.

Test pierwszy - na średniej wielkości instancji

Ilość sekwencji w instancji: 20,

długość sekwencji: 50,

długość sekwencji optymalnej: 80,

ilość błędów w instancji: 0.

Brak dynamicznej zmiany wartości prawdopodobieństwa występowania mutacji.

Rozmiar populacji	Liczba iteracji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
50	100	97,5	7,5864
	200	96,3	12,7485
	300	94,5	16,4618
	400	99,5	19,23
	500	94,4	23,4266

100	200	88,5	28,531
	300	87,9	34,4278
	400	86,3	38,4744
	500	87,8	48,4786
	600	88,1	53,3628
200	100	83,9	49,5839
	200	82,8	62,2537
	300	85,9	79,4838
	400	83,3	89,9902
	500	83,9	122,3037
300	100	82,6	106,0014
	200	83,1	121,6848
	300	83,7	142,0619
	400	81,7	148,6505
	500	82	165,4222
400	100	81,7	115,9949

Tabela 5.1 - Wyniki testów na średniej wielkości instancji.

Można zauważyć, że zarówno zwiększanie liczby iteracji, jak i rozmiaru populacji wpływa na poprawę jakości uzyskiwanych rozwiązań, jednak zależność ta nie jest liniowa. Dla najmniejszych populacji (50 osobników) średnia wartość funkcji celu utrzymuje się na poziomie około 94–100, natomiast zwiększenie rozmiaru populacji do 100 osobników pozwala obniżyć ją do około 86–89. Dalsze zwiększanie populacji do 200–400 osobników daje już znacznie mniejsze korzyści, a poprawa wyników jest niewielka w stosunku do wzrostu czasu działania algorytmu.

Wpływ liczby iteracji jest zauważalny głównie dla mniejszych populacji. Przykładowo dla populacji 50 osobników zwiększenie liczby iteracji ze 100 do 300 poprawiło średnią wartość funkcji celu z 97,5 do 94,5. Dla większych populacji poprawa jest już znacznie mniejsza, co sugeruje, że algorytm stosunkowo szybko osiąga obszar dobrych rozwiązań. Jednocześnie czas działania rośnie proporcjonalnie zarówno do liczby iteracji, jak i rozmiaru populacji.

Najlepszy średni wynik uzyskano dla populacji 400 osobników i 100 iteracji oraz dla populacji 300 osobników i 400 iteracji (średnia wartość funkcji celu równa 81,7). Biorąc jednak pod uwagę czas działania algorytmu, najbardziej opłacalnym wyborem dla instancji o 20 sekwencjach długości 50 wydaje się populacja około 200–300 osobników oraz 100–200 iteracji. Parametry te zapewniają wyniki bardzo zbliżone do najlepszych uzyskanych w eksperymencie przy znacznie krótszym czasie obliczeń niż konfiguracje wykorzystujące największe populacje i liczbę iteracji.

Test drugi - wpływ błędów na jakość rozwiązania

Ilość sekwencji w instancji: 20,

długość sekwencji: 50,

długość sekwencji optymalnej: 80,

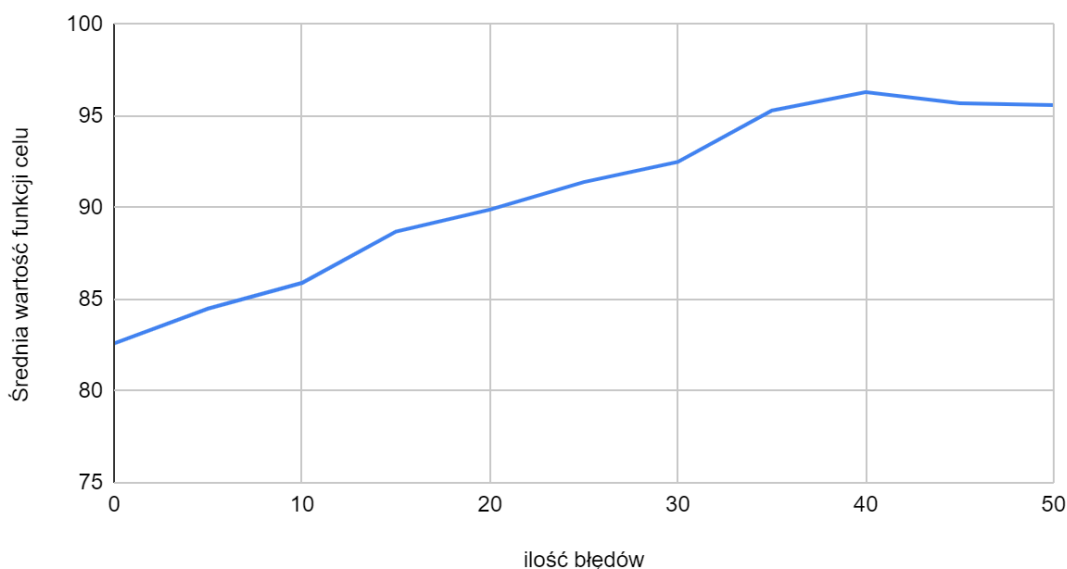
ilość błędów w instancji: testowana,
 dynamiczna zmiana wartości prawdopodobieństwa występowania mutacji,
 rozmiar populacji: 300,
 liczba iteracji: 100.

Parametry ilość iteracji i rozmiar populacji zostały wybrane na podstawie testu pierwszego. Wpływ błędów testowany jest na instancjach o identycznych parametrach (poza liczbą błędów) co w przypadku testu pierwszego. Została też odblokowana dynamiczna zmiana wartości prawdopodobieństwa występowania mutacji. Błędy nie są wprowadzane do tych samych instancji, za każdym razem przy zmianie ich ilości tworzone jest 10 nowych instancji.

Ilość błędów w instancji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
0	82,6	89,0583
5	84,5	88,5723
10	85,9	93,0504
15	88,7	104,3306
20	89,9	107,2161
25	91,4	115,2478
30	92,5	123,4402
35	95,3	132,8024
40	96,3	127,0226
45	95,7	141,2397
50	95,6	159,6151

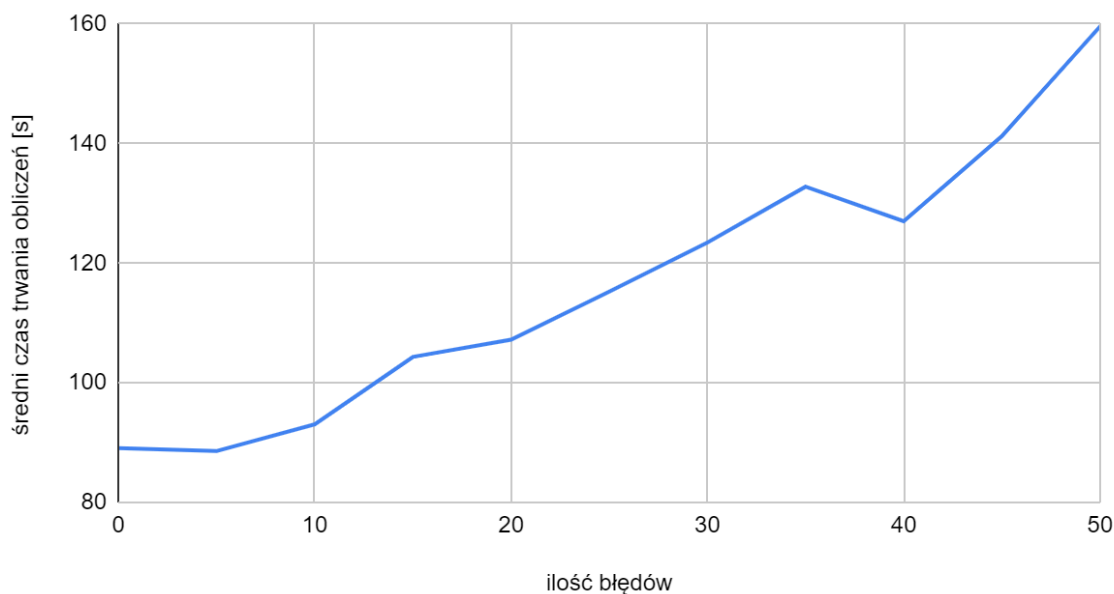
Tabela 5.2 - Wyniki testów różnych ilości błędów w instancji.

Zależność funkcji celu od ilości błędów w instancji



Wykres 5.1 - zależność funkcji celu od ilości błędów w instancji - na podstawie Tabeli 5.2.

Zależność czasu trwania obliczeń od ilości błędów w instancji



Wykres 5.2 - zależność czasu trwania obliczeń metaheurystyki od ilości błędów w instancji - na podstawie Tabeli 5.2.

Wyniki testu pokazują wyraźny wpływ liczby błędów w instancji zarówno na jakość uzyskiwanych rozwiązań, jak i czas działania algorytmu. Wraz ze wzrostem liczby błędów rośnie średnia wartość funkcji celu, co oznacza pogorszenie jakości znajdowanych rozwiązań. Dla instancji bez błędów średnia wartość funkcji celu wyniosła 82,6, natomiast dla instancji zawierających 50 błędów wzrosła do około 95,6. Oznacza to, że dodatkowe błędy znacząco utrudniają odnalezienie krótkiego wspólnego dopasowania dla wszystkich sekwencji, albo poprawniej - ich obecność zwykle znacząco wydłuża istniejące optymalne dopasowanie sekwencji. Wzrost liczby błędów powoduje również wydłużenie czasu działania algorytmu. Średni czas obliczeń zwiększył się z około 89 sekund dla instancji bez błędów do prawie 160 sekund dla instancji zawierających 50 błędów.

Test trzeci - na większej instancji

Ilość sekwencji w instancji: 30,

długość sekwencji: 60,

długość sekwencji optymalnej: 90,

ilość błędów w instancji: 10.

Dynamiczna zmiana wartości prawdopodobieństwa występowania mutacji.

Rozmiar populacji	Liczba iteracji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
50	100	119,6	15,4744
	200	111,4	23,4112
	300	109,8	26,7088
	400	108,8	32,392

50

	500	106,3	36,937
100	100	106,8	38,627
	200	100,6	53,4135
	300	101,1	59,7579
	400	103,3	71,4033
	500	99,6	82,6892
200	100	96,3	92,2412
	200	99,8	114,319
	300	98,1	144,1073
	400	96,4	161,654
	500	96,5	177,2926
300	100	98,1	164,1601
	200	96,3	186,7713
	300	96,5	231,2939
	400	97,7	257,7405
	500	95,9	288,5494
400	100	97	203,3281

Tabela 5.3 - Wyniki testów na większej instancji.

Najlepsze wyniki uzyskano dla populacji 300 osobników oraz 500 iteracji, jednak różnice w porównaniu do wartości np. 300 i 200 lub 200 i 100 są niewielkie. Uwzględniając czas działania algorytmu, za najbardziej korzystne można uznać populację około 200–300 osobników oraz 100–200 iteracji (czas dla takich wartości parametrów będzie prawie dwa razy krótszy niż dla 300 i 500). Są to więc podobne wartości co w przypadku testu pierwszego, mimo większej instancji.

Widać również, że mała liczba błędów nie powoduje drastycznie większej wartości funkcji celu.

Test czwarty - na mniejszej instancji

Ilość sekwencji w instancji: 15,

długość sekwencji: 40,

długość sekwencji optymalnej: 60,

ilość błędów w instancji: 25.

Dynamiczna zmiana wartości prawdopodobieństwa występowania mutacji.

Rozmiar populacji	Liczba iteracji	Średnia wartość funkcji celu	Średni czas działania algorytmu [s]
50	100	74,1	4,4798
50	200	71,9	6,2204
50	300	72,7	9,6907

50	400	70,8	12,0689
50	500	71,6	14,834
100	100	70,2	13,4591
100	200	69,9	17,4465
100	300	70,9	23,2418
100	400	69,9	23,9285
100	500	70,4	32,4042
200	100	69,3	25,3938
200	200	68,6	37,3558
200	300	69,1	49,7688
200	400	69,2	57,3801
200	500	68,7	68,9901
300	100	68,9	44,271
300	200	69	64,0617
300	300	68,7	80,1525
300	400	68,9	101,0711
300	500	68,9	115,6426
400	100	68,8	67,5722

Tabela 5.4 - Wyniki testów na mniejszej instancji.

W porównaniu do poprzednich eksperymentów dobre wyniki osiągnąć się już dla mniejszych populacji i mniejszej liczby iteracji. Wynika to najprawdopodobniej wprost z mniejszego rozmiaru instancji (chodzi głównie o krótsze sekwencje w instancji), co sprawia, że przestrzeń rozwiązań jest łatwiejsza do przeszukania mimo większej liczby błędów w wygenerowanych sekwencjach (warto też pamiętać, że błędy nie muszą powodować wydłużenia wydłużenia optymalnego dopasowania - może ono pozostać podobnej wielkości, a w rzadkich sytuacjach nawet zostać skrócone).

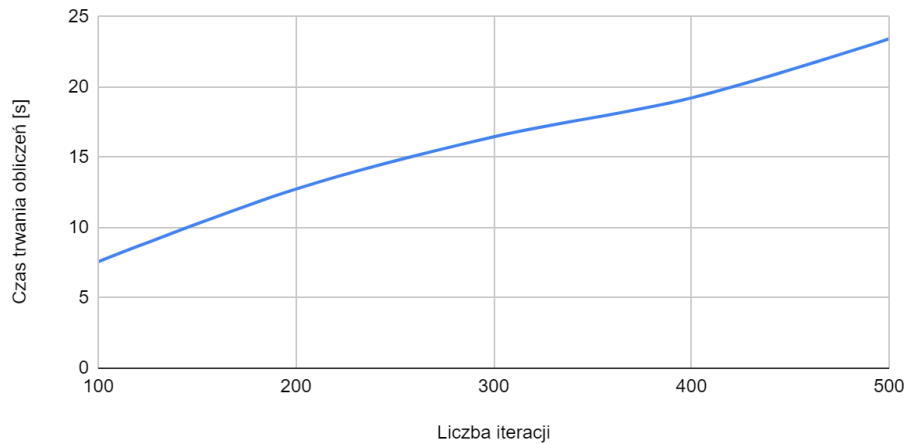
Podobnie jak w poprzednich testach, zwiększanie rozmiaru populacji oraz liczby iteracji prowadzi do poprawy jakości uzyskiwanych rozwiązań, jednak zależność ta jest stosunkowo słaba. Najlepszy średni wynik uzyskano dla populacji 200 osobników oraz 200 iteracji, natomiast dalsze zwiększanie liczby iteracji lub rozmiaru populacji nie przynosi już zauważalnej poprawy jakości rozwiązania. Różnice pomiędzy najlepszym i najgorszym wynikiem dla populacji 200-400 osobników nie przekraczają jednej jednostki funkcji celu, podczas gdy czas działania wzrasta nawet kilkakrotnie.

Uwzględniając zarówno jakość rozwiązań, jak i czas działania algorytmu, najbardziej korzystne dla tego typu instancji wydają się populacje o rozmiarze około 100-200 osobników oraz 100-200 iteracji. W przypadku tej instancji zwiększanie parametrów ponad te wartości nie znajduje uzasadnienia praktycznego, ponieważ uzyskiwana poprawa jest niewielka w porównaniu do dodatkowego czasu obliczeń.

Wykresy zależności czasu trwania obliczeń i funkcji celu od liczby iteracji i rozmiaru populacji

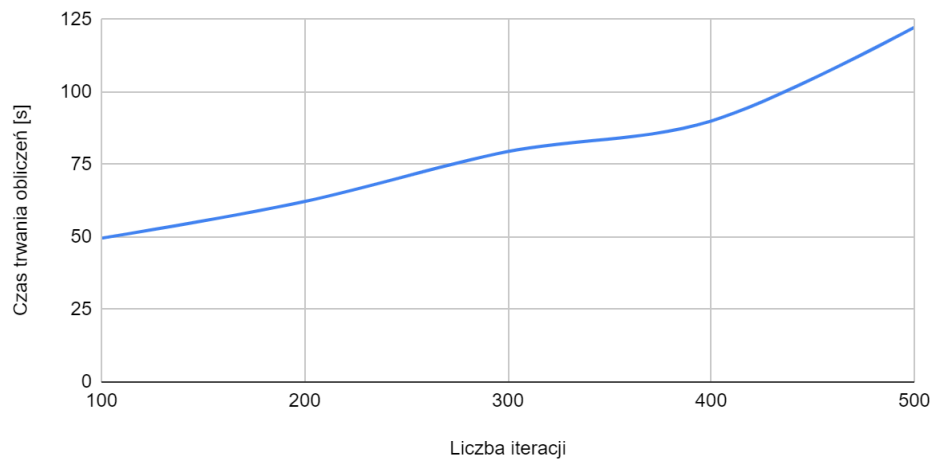
Na podstawie danych z testu pierwszego:

Wykres zależności czasu trwania obliczeń od parametru "ilość iteracji" dla rozmiaru populacji 50



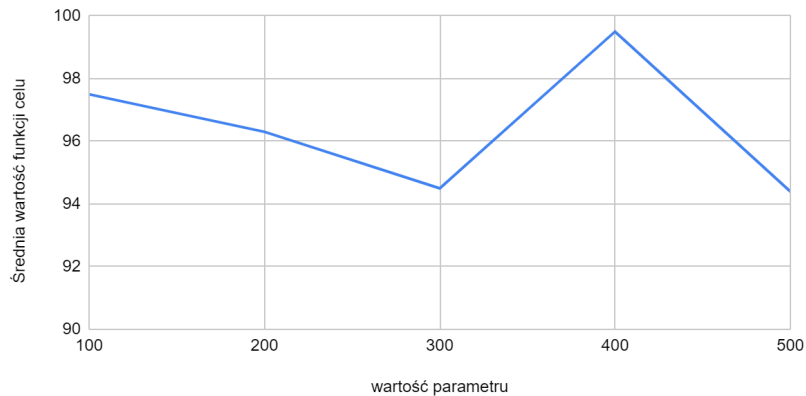
Wykres 5.3 - zależność czasu trwania obliczeń metaheurystyki od parametru "ilość iteracji" - na podstawie Tabeli 5.1.

Wykres zależności czasu trwania obliczeń od parametru "ilość iteracji" dla rozmiaru populacji 200



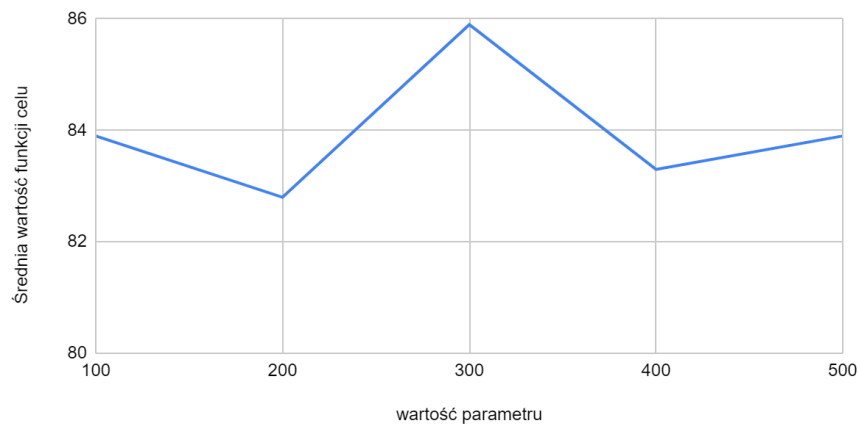
Wykres 5.4 - zależność czasu trwania obliczeń metaheurystyki od parametru "ilość iteracji" - na podstawie Tabeli 5.1.

Wykres zależności funkcji celu od parametru "ilość iteracji" dla rozmiaru populacji 50



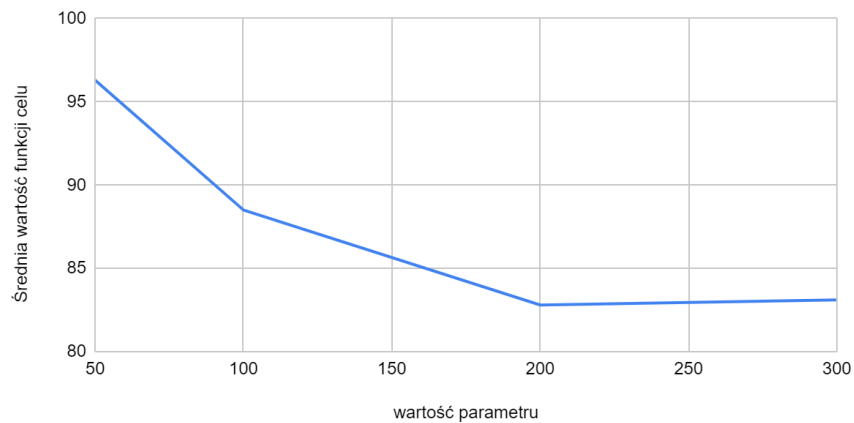
Wykres 5.5 - zależność funkcji celu od parametru "ilość iteracji" - na podstawie Tabeli 5.1.

Wykres zależności funkcji celu od parametru "ilość iteracji" dla rozmiaru populacji 200



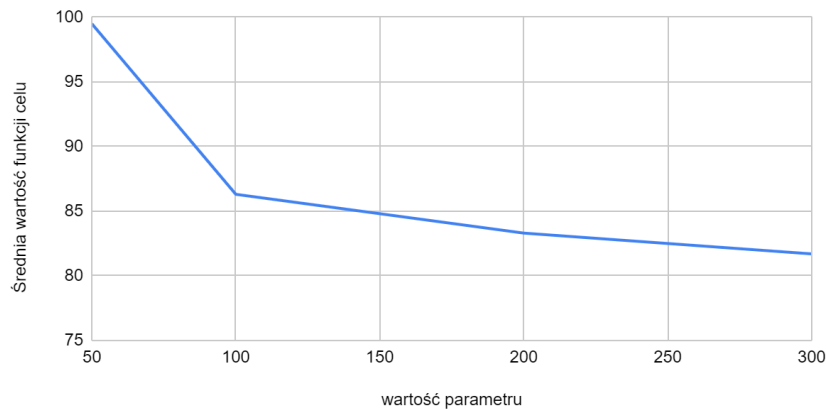
Wykres 5.6 - zależność funkcji celu od parametru "ilość iteracji" - na podstawie Tabeli 5.1.

Wykres zależności funkcji celu od parametru "rozmiar populacji" dla liczby iteracji 200



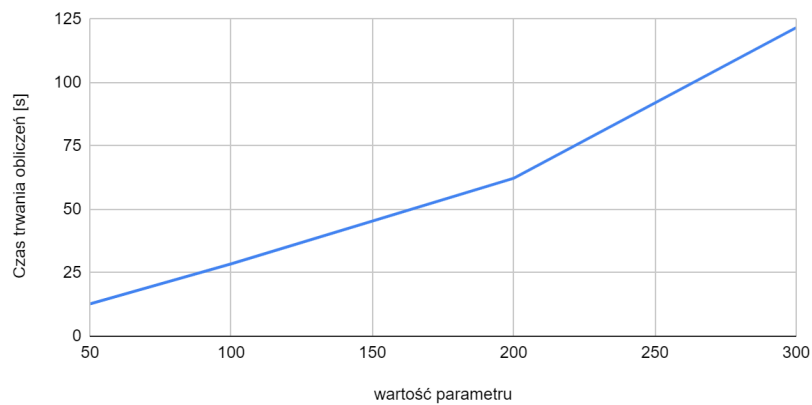
Wykres 5.7 - zależność czasu trwania obliczeń metaheurystyki od parametru "rozmiar populacji" - na podstawie Tabeli 5.1.

Wykres zależności funkcji celu od parametru "rozmiar populacji" dla liczby iteracji 400



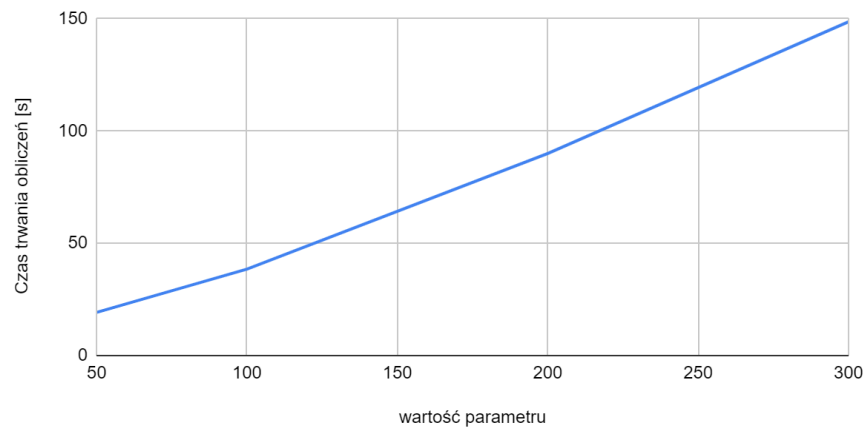
Wykres 5.8 - zależność czasu trwania obliczeń metaheurystyki od parametru "rozmiar populacji" - na podstawie Tabeli 5.1.

Wykres zależności czasu trwania obliczeń metaheurystyki od parametru "rozmiar populacji" dla liczby iteracji 200



Wykres 5.9 - zależność funkcji celu od parametru "rozmiar populacji" - na podstawie Tabeli 5.1.

Wykres zależności czasu trwania obliczeń metaheurystyki od parametru "rozmiar populacji" dla liczby iteracji 400



Wykres 5.10 - zależność funkcji celu od parametru "rozmiar populacji" - na podstawie Tabeli 5.1.

Zbiorcze wnioski:

Na wykresach zależności czasu działania od liczby iteracji można zauważyć niemal liniowy wzrost czasu obliczeń. Zależność ta występuje niezależnie od rozmiaru populacji i wynika bezpośrednio z konstrukcji algorytmu, który w każdej iteracji wykonuje zbliżoną liczbę operacji. Wpływ liczby iteracji na czas działania jest więc bardzo regularny i łatwy do przewidzenia.

Znacznie trudniej wyciągnąć jednoznaczne wnioski z wykresów przedstawiających zależność wartości funkcji celu od liczby iteracji. Zwiększenie liczby iteracji zwykle prowadzi do pewnej poprawy jakości rozwiązania, ale zależność ta nie jest regularna/stabilna. Różnice pomiędzy kolejnymi wartościami parametru liczby iteracji są często niewielkie i mogą być mocno zaburzone przez losowość algorytmu genetycznego.

Na wykresach zależności czasu działania od rozmiaru populacji również obserwowany jest wzrost zbliżony do liniowego, jest on wyraźnie silniejszy niż w przypadku liczby iteracji. Wynika to ze złożoności obliczeniowej metaheurystyki. Ponieważ powiększenie populacji powoduje wzrost liczby osobników uczestniczących w selekcji, reprodukcji i mutacjach, przez co czas pojedynczej iteracji znacząco rośnie. W praktyce więc zmiana rozmiaru populacji ma większy wpływ na całkowity czas działania algorytmu niż zmiana liczby iteracji. Wykresy przedstawiające wpływ rozmiaru populacji na wartość funkcji celu przypominają malejące funkcje wykładnicze (w szczególności wykres dla liczby iteracji 200). Wraz ze wzrostem liczebności populacji średnia wartość funkcji celu maleje, jednak tempo tej poprawy stopniowo się zmniejsza. Oznacza to, że początkowe zwiększanie populacji przynosi znacznie lepsze wartości funkcji celu, natomiast dla dużych populacji kolejne przyrosty liczebności dają coraz mniejsze efekty. Sugeruje to występowanie dla każdej instancji punktu, po przekroczeniu którego dalsze zwiększanie populacji staje się mało opłacalne ze względu na szybko rosnący czas obliczeń i niewielką poprawę funkcji celu rozwiązań.

6. Złożoność obliczeniowa i pamięciowa

Złożoność obliczeniowa metaheurystyki projektowanej w ramach niniejszego projektu jest silnie zależna od wartości parametrów instancji i metaheurystyki oraz od przebiegu mechanizmów z niepodaną wprost liczbą wywołań, przede wszystkim reprodukcji. W szczególności znaczenie mają: liczba sekwencji w instancji (m), długość sekwencji (n), średnia długość osobników w populacji w trakcie działania algorytmu (L), liczebność populacji (P) oraz liczba iteracji algorytmu (i). Dodatkowo część operacji wykonywana jest losowo, przez co rzeczywista liczba wywołań niektórych funkcji może różnić się pomiędzy uruchomieniami nawet dla tych samych parametrów.

Generowanie populacji początkowej wymaga utworzenia P osobników. Każdy z nich budowany jest znak po znaku aż do uzyskania rozwiązania dopuszczalnego. Dla każdego dodawanego nukleotydu sprawdzane jest jego dopasowanie do wszystkich sekwencji instancji, co daje złożoność rzędu $O(P \cdot L \cdot m)$.

Najbardziej złożonym czasowo elementem selekcji jest sortowanie rodziców. Przy każdym losowaniu rodzica populacja jest sortowana według funkcji celu (długości chromosomów), co wymaga $O(P \cdot \log P)$ operacji. Ponieważ w trakcie jednej iteracji wybór rodzica wykonywany jest wielokrotnie (zarówno przy wyznaczaniu osobników przeżywających, jak i podczas tworzenia potomków), całkowity koszt selekcji można oszacować na $O(P^2 \cdot \log P)$ na jedną iterację algorytmu.

Mechanizm reprodukcji rozpoczyna się od wyznaczenia najdłuższego wspólnego podciągu (LCS) dwóch rodziców. Wykorzystany algorytm programowania dynamicznego posiada złożoność $O(L^2)$ czasową (jak i również pamięciową). Po znalezieniu podciągu tworzony jest chromosom potomka oraz wykonywany jest mechanizm naprawczy sprawdzający jego dopuszczalność względem wszystkich sekwencji instancji. Ta część wymaga około $O(L \cdot m + k \cdot m)$ operacji, gdzie k oznacza maksymalną liczbę prób wydłużenia potomka. Łącznie koszt pojedynczego krzyżowania wynosi więc $O(L^2 + L \cdot m + k \cdot m)$.

Ważnym czynnikiem, wpływającym na złożoność jest fakt, że liczba wywołań procedury reprodukcji nie jest stała. W każdej iteracji algorytm tworzy potomków, aż do osiągnięcia docelowej liczebności populacji. Ponieważ część potomków może zostać odrzucona (gdy nie uda się uzyskać rozwiązania dopuszczalnego w wyznaczonym limicie prób wydłużenia), rzeczywista liczba wywołań funkcji reprodukcji zależy w pełni od wyników losowych wyborów pojawiających się podczas działania funkcji. W praktyce można przyjąć, że liczba prób wywołania funkcji jest wprost proporcjonalna do liczebności populacji, jednak jest to luźne i niepewne oszacowanie, dodatkowo nieznane są konkretne średnie zależności od liczebności populacji. Na potrzeby oszacowania złożoności obliczeniowej przyjęte zostało, że jest to zależność liniowa.

Kolejnym etapem są mutacje. Liczba wykonywanych mutacji zależy od sumarycznej długości wszystkich osobników w populacji oraz od parametru prawdopodobieństwa mutacji. W przybliżeniu wykonywanych jest około $p \cdot P \cdot L$ mutacji, gdzie p to prawdopodobieństwo mutacji. Każda mutacja wymaga sprawdzenia dopasowania wszystkich sekwencji instancji do nowego rozwiązania oraz ponownego uproszczenia rozwiązania, co oznacza złożoność obliczeniową dla jednej mutacji około $O(L \cdot m)$. W rezultacie całkowita złożoność mechanizmu mutacji w jednej iteracji może zostać oszacowana jako $O(p \cdot P \cdot L^2 \cdot m)$.

Pozostałe operacje, takie jak aktualizacja najlepszego znalezionej rozwiązania, sprawdzanie przestoju algorytmu czy aktualizacja danych prezentowanych w interfejsie użytkownika, mają znacznie mniejszy wpływ na całkowity czas działania. Ich złożoność obliczeniowa jest pomijalna lub zależna od liczebności populacji (aktualizacja najlepszego rozwiązania), co można oszacować jako $O(P)$ na iterację.

Uwzględniając wszystkie opisane wcześniej elementy, złożoność czasową pojedynczej iteracji algorytmu można przedstawić w postaci: $O(P^2 \cdot \log P + P \cdot (L^2 + L \cdot m + k \cdot m) + p \cdot P \cdot L^2 \cdot m + P)$, a cały przebieg działania algorytmu genetycznego, obejmujący generowanie populacji początkowej oraz i iteracji: **$O(P \cdot L \cdot m + i \cdot (P^2 \cdot \log P + P \cdot (L^2 + L \cdot m + k \cdot m) + p \cdot P \cdot L^2 \cdot m + P))$** .

W praktyce największy wpływ na czas działania algorytmu mają proces selekcji rodziców oraz reprodukcja potomków. Reprodukacja wykorzystuje algorytm wyznaczania najdłuższego wspólnego podciągu (LCS), którego złożoność jest kwadratowa względem długości chromosomów rodziców. Dodatkowo selekcja rankingowa wymaga sortowania populacji podczas losowania rodziców, co przy większych populacjach generuje zauważalny koszt obliczeniowy. Operacje mutacji chociaż cechują się dużą złożonością obliczeniową, mają w praktyce stosunkowo niewielki wpływ na całkowity czas działania algorytmu, ponieważ po strojeniu parametrów prawdopodobieństwo mutacji przyjmuje bardzo małe wartości i liczba wykonywanych mutacji w pojedynczej iteracji jest niewielka. Złożoność pamięciowa algorytmu wynika głównie z przechowywania populacji oraz macierzy wykorzystywanej podczas obliczania LCS i wynosi w oszacowaniu $O(P \cdot L + L^2 + m)$.